

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_e5140909-890\agent-af05a23e49becf039.jsonl`  
- SHA-256: `f3e60ae3ca4693753a7c4717278a66a910f9d6875ac6ac7377410f936a500175`  
- Source modified: `2026-06-18T07:51:01+00:00`  
- Imported at: `2026-07-05T16:48:26+00:00`  
- project: `wf_e5140909-890`  
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T07:47:49.196Z)

Work READ-ONLY in this git worktree pinned to origin/master (DO NOT modify):

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/refactor-base  
Read the ACTUAL code (and the matching tests/ file) before asserting anything; never guess.  
Adversarial bug-hunter. Report ONLY real, reachable defects (crash on a reachable input, wrong  
logic, inverted condition, bad error handling, resource leak, off-by-one, contract mismatch,  
unguarded external/persisted-data access). Exact file+line + concrete trigger + minimal fix +  
confidence. PRECISION over recall ? empty list is a respected answer; do NOT invent findings.  
The repo has ~855 passing tests + several hardening sweeps; expect few. Skip pure style.  
AREA: scripts/ (incl. nightly_regression.py now ? it's in scope for this sweep),  
backend/reporting/, backend/tools/, training/. argparse defaults, exit codes, file/path  
handling, report assembly.
```

2. assistant (2026-06-18T07:47:52.085Z)

I'll start by exploring the directory structure for the areas in scope, then read the actual code carefully.

3. user (2026-06-18T07:47:53.604Z)

```
a010667 feat(eval): nightly all-OFF floor ? pin a baseline_off reference config (#217)
```

```
---
```

```
backend/reporting/:
```

```
__init__.py  
harvest.py  
run_signals.py  
spec.yaml
```

```
backend/tools/:
```

```
__init__.py  
export.py  
rally_slicer.py
```

```
scripts/:
```

```
README.md  
doctor.py  
nightly_regression.ps1  
nightly_regression.py  
register_nightly_task.ps1  
run_phase3_eval.py  
run_process_and_stitch.py
```

```
training/:
```

```
README.md  
__init__.py
```

gen0
requirements-train.txt

4. assistant (2026-06-18T07:47:55.507Z)

Let me read all the in-scope files. Starting with the scripts.

5. user (2026-06-18T07:47:56.894Z)

```
1      """Nightly rally-quality regression tripwire.
2
3      Re-scores the **golden corpus** under HEAD via :mod:`backend.eval.rally_seg_eval` (CPU,
4      seconds ? the WASB GPU trajectories are CACHED, only the windowing/scorer changes) and
5      diffs
6      the headline metrics against a FROZEN baseline
7      (`eval_baselines/nightly_baseline.json`).
8      Writes a dated report and exits non-zero on any regression beyond tolerance OR any per-
9      video
10     over-segmentation-guard regression.
11
12     This is the owner's dual-eval-set discipline made automatic (`memory/eval-dual-set-
13     regression`
14     + ``docs/R2_EVAL_METRIC_DESIGN.md`` $4.5): the growing GOLDEN-finalized set is re-run
15     every
16     night so a *late* change cannot silently lose an *earlier* win. It is **default-
17     ADDITIVE** ? it
18     does NOT change the promotion gate. It is a *tripwire*, not a re-training run: no GPU,
19     no Gemini,
20     no WASB re-extraction (those are cached and unchanged night to night).
21
22     Three tracked configs (all re-scored every night; a clean A/B/C on the same SERVED base
23     knobs):
24
25     * **DEFAULT** ? the SERVED production windowing, i.e. what users get today. Catches a
26     regression
27     to shipped behaviour. **NOTE (post-#204, 2026-06-18):** the R1 milestone is now
28     flipped ON, so
29     SERVED *is* cap=6 + R4 ? "default" now equals "milestone".
30     * **MILESTONE** ? the explicit R3+R4 config (cap=6 s + rally-END inactivity trim;
31     ``docs/QUALITY_ITERATIONS.md`` R3/R4). Now == DEFAULT (production shipped it); kept
32     explicit so
33     the nightly still names the milestone knobs and stays meaningful if production later
34     diverges.
35     * **BASELINE_OFF** ? the pre-R1-flip all-OFF windowing (every quality knob forced OFF).
36     The
37     permanent historical floor, so each report keeps showing the delta the milestone buys
38     (a launched
39     win must keep standing its ground, not be taken at launch-time value).
40
41     Ground truth + trajectories are LOCAL (the manifest points at owned, minors-free
42     trajectory CSVs
43     outside the repo + ``output/<name>_golden_features.json``); the corpus is NOT committed.
44     So the
45     baseline is tied to the owner's local golden corpus and is regenerated with ``--update-
46     baseline``
47     whenever the corpus grows or an intended improvement lands. (A cloud agent does not
48     suffice ? it
49     has the repo but not the cached trajectories; hence the local Windows Scheduled Task
50     wiring in
51     ``scripts/nightly_regression.ps1``.)
52
53     Usage
54     -----
55     Snapshot the baseline from current HEAD (after an intended improvement or a corpus
56     change):
```

```

37
38     python scripts/nightly_regression.py --update-baseline
39
40     Nightly check (exit 1 on regression; writes ``output/nightly_regression/<date>.md`` +
41     ``.json``):
42
43     python scripts/nightly_regression.py
44
45     Exit codes
46     -----
47     * ``0`` ? all tracked configs clean (no regression), corpus matches the baseline.
48     * ``1`` ? at least one regression (an aggregate metric dropped beyond tolerance, or a
49     per-video
50     over-seg guard increased). Takes precedence over a stale baseline.
51     * ``2`` ? operational error (manifest/corpus missing, nothing scored).
52     * ``3`` ? no baseline to compare against (snapshot one with ``--update-baseline``).
53     * ``4`` ? baseline STALE (the corpus set or a config definition changed): a refresh is
54     needed,
55     but no regression was found on the comparable intersection.
56     """
57     from __future__ import annotations
58
59     import argparse
60     import datetime as _dt
61     import json
62     import os
63     import sys
64     from dataclasses import dataclass, field, replace
65     from typing import Any, Dict, List, Optional, Tuple
66
67     # ----- #
68     # Paths (resolved relative to the repo root, so the script is cwd-robust).
69     # ----- #
70     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
71     if REPO_ROOT not in sys.path:
72         sys.path.insert(0, REPO_ROOT) # importable as `python
73     scripts/nightly_regression.py` from anywhere
74     DEFAULT_BASELINE = os.path.join(REPO_ROOT, "eval_baselines", "nightly_baseline.json")
75     DEFAULT_REPORT_DIR = os.path.join(REPO_ROOT, "output", "nightly_regression")
76     DEFAULT_MANIFEST = os.path.join(REPO_ROOT, "output", "human_lovo_manifest.json")
77     DEFAULT_FEATURES_DIR = os.path.join(REPO_ROOT, "output")
78     DEFAULT_IOU = 0.5
79
80     # ----- #
81     # What we gate on (a tripwire ? focused, not every metric).
82     # * HIGHER-better aggregates regress when they DROP beyond ``metric_tol``.
83     # * merge_rate (lower-better) regresses when it RISES beyond ``rate_tol``.
84     # * per-video split_count / merge_count must NOT increase beyond ``over_seg_tol``
85     # (the R2 §2.3.3 guard: "may not increase EITHER on ANY video").
86     # Boundary-error medians + seg_ratio are outlier-dominated diagnostics ? REPORTED, not
87     gated.
88     # ----- #
89     GATED_HIGHER: Tuple[str, ...] = ("tol_f1", "f1@0.5", "f1@0.25")
90     GATED_LOWER: Tuple[str, ...] = ("merge_rate",)
91     PER_VIDEO_GUARD: Tuple[str, ...] = ("split_count", "merge_count")
92
93     #: Aggregate metrics snapshotted into the baseline (the gated subset + report-only
94     context).
95     AGG_SNAPSHOT: Tuple[str, ...] = (
96         "tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision", "recall",
97         "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median",
98         "dend_abs_median",
99     )
100     #: Per-video metrics snapshotted into the baseline.
101     PV_SNAPSHOT: Tuple[str, ...] = (

```

```

95         "tol_f1", "f1@0.5", "f1@0.25", "merge_rate", "split_count", "merge_count",
96         "num_pred", "num_gt",
97     )
98     DEFAULT_METRIC_TOL = 0.005    # F1-like drop that counts as a regression (absorbs float
jitter)
99     DEFAULT_RATE_TOL = 0.010      # merge_rate rise that counts as a regression
100    DEFAULT_OVER_SEG_TOL = 0       # per-video split/merge increase allowed (strict)
101
102
103    # ----- #
104    # The two tracked configs.
105    # ----- #
106    @dataclass(frozen=True)
107    class NightlyConfig:
108        """A named windowing config the nightly re-scores (preset + net-crossings gate)."""
109
110        name: str
111        preset: Any          # backend.eval.windowing.WindowingPreset
112        min_crossings: int = 0
113        note: str = ""
114
115
116    def tracked_configs() -> List[NightlyConfig]:
117        """DEFAULT (SERVED = shipped) + MILESTONE (explicit cap=6 + R4) + BASELINE_OFF (the
pre-flip floor).
118
119        All three are anchored on the SAME ``windowing.SERVED`` base knobs (velocity_thresh
/ merge_gap /
120        min_window) so they differ ONLY in the quality knobs ? they are a clean A/B/C:
121
122        * **DEFAULT** ? ``SERVED`` verbatim, so it can never silently drift from the served
operating
123        point. **Post-#204 (2026-06-18) the R1 milestone is flipped ON ? SERVED IS cap=6 +
R4, so
124        DEFAULT now equals MILESTONE.** Its job: catch an *accidental* change to a
production windowing
125        knob.
126        * **MILESTONE** ? SERVED + the explicit R3/R4 knobs (``docs/QUALITY_ITERATIONS.md``
R3/R4). Pins
127        the milestone *definition* independent of what production ships, so "is cap=6 + R4
still scoring
128        what we expect?" stays answerable even if a future change moves DEFAULT.
129        * **BASELINE_OFF** ? SERVED with every quality knob forced OFF (the pre-R1-flip
windowing). Pins
130        the *historical floor* so each nightly report keeps showing the delta the
milestone buys (the
131        dual-eval-set discipline: a launched win must keep standing its ground, not be
taken at
132        launch-time value). It is the reference DEFAULT used to be before the flip."""
133        from backend.eval.windowing import SERVED
134
135        default = NightlyConfig(
136            name="default", preset=SERVED, min_crossings=0,
137            note="SERVED production windowing ? now the R1 milestone (cap=6 + R4) after the
#204 flip.",
138        )
139        milestone = NightlyConfig(
140            name="milestone",
141            preset=replace(SERVED, name="milestone", max_window_duration=6.0,
142                          inactivity_end=1.5, inactivity_rally_thresh=0.20,
143                          inactivity_min_density=0.30),
144            min_crossings=0,
145            note="R3 cap=6 s + R4 rally-END inactivity trim ? the shipped milestone (==
default post-#204).",

```

```

146         )
147         baseline_off = NightlyConfig(
148             name="baseline_off",
149             preset=replace(SERVED, name="baseline_off", max_window_duration=0.0,
hard_cap=False,
150                             inactivity_end=0.0, inactivity_rally_thresh=0.08,
inactivity_min_density=0.34,
151                             blob_fallback=False, blob_factor=4.0),
152             min_crossings=0,
153             note="Pre-R1-flip all-OFF windowing ? the historical floor; pins the value the
milestone buys.",
154         )
155         return [default, milestone, baseline_off]
156
157
158         # ----- #
159         # Tolerances bundle.
160         # ----- #
161         @dataclass(frozen=True)
162         class Tolerances:
163             metric_tol: float = DEFAULT_METRIC_TOL
164             rate_tol: float = DEFAULT_RATE_TOL
165             over_seg_tol: int = DEFAULT_OVER_SEG_TOL
166
167             def to_dict(self) -> Dict[str, Any]:
168                 return {"metric_tol": self.metric_tol, "rate_tol": self.rate_tol,
169                     "over_seg_tol": self.over_seg_tol}
170
171
172         # ----- #
173         # Pure summary + comparison core (no I/O ? unit-testable).
174         # ----- #
175         def summarize_run(eval_result: Dict[str, Any]) -> Dict[str, Any]:
176             """Compact, comparable summary of one :func:`rally_seg_eval.evaluate` result.
177
178             Keeps only the deterministic quantities we compare ? aggregate MEANS (not the
bootstrap CI,
179             which is seeded but irrelevant to the gate) and per-video RAW metric values."""
180             agg = eval_result.get("aggregate", {}) or {}
181             aggregate = {k: (agg[k]["mean"] if isinstance(agg.get(k), dict) else None) for k in
AGG_SNAPSHOT}
182             per_video = {r["name"]: {k: r[k] for k in PV_SNAPSHOT if k in r} for r in
eval_result.get("rows", [])}
183             return {
184                 "n": int(agg.get("n", len(eval_result.get("rows", [])))),
185                 "preset": eval_result.get("preset"),
186                 "min_crossings": eval_result.get("min_crossings", 0),
187                 "iou": eval_result.get("iou", DEFAULT_IOU),
188                 "aggregate": aggregate,
189                 "per_video": per_video,
190             }
191
192
193         @dataclass
194         class Finding:
195             """One comparison outcome line (regression / improvement / stale note)."""
196
197             config: str
198             scope: str          # "aggregate" | "per_video" | "config"
199             metric: str
200             kind: str           # "regression" | "improvement" | "stale"
201             baseline: Optional[float] = None
202             current: Optional[float] = None
203             video: Optional[str] = None
204

```

```

205     @property
206     def delta(self) -> Optional[float]:
207         if self.baseline is None or self.current is None:
208             return None
209         return self.current - self.baseline
210
211     def message(self) -> str:
212         loc = f"{self.config}/{self.video}/{self.metric}" if self.video else
f"{self.config}/{self.metric}"
213         if self.kind == "stale":
214             return f"[STALE] {self.config}: {self.metric}"
215         d = self.delta
216         ds = f"{d:+.4f}" if d is not None else "?"
217         b = "?" if self.baseline is None else f"{self.baseline:.4f}"
218         c = "?" if self.current is None else f"{self.current:.4f}"
219         tag = "REGRESSION" if self.kind == "regression" else "improvement"
220         return f"[{tag}] {loc}: {b} ? {c} ({ds})"
221
222
223     @dataclass
224     class ConfigComparison:
225         config: str
226         status: str = "ok"          # "ok" | "regression" | "stale"
227         findings: List[Finding] = field(default_factory=list)
228         added_videos: List[str] = field(default_factory=list)
229         removed_videos: List[str] = field(default_factory=list)
230         aggregate_gated: bool = True # False when preset/corpus changed ? aggregate not
comparable
231
232     @property
233     def regressions(self) -> List[Finding]:
234         return [f for f in self.findings if f.kind == "regression"]
235
236     @property
237     def improvements(self) -> List[Finding]:
238         return [f for f in self.findings if f.kind == "improvement"]
239
240
241     def _preset_comparable(base_preset: Any, cur_preset: Any) -> bool:
242         """Are the two stored preset dicts the same windowing definition? (name is
cosmetic)."""
243         if not isinstance(base_preset, dict) or not isinstance(cur_preset, dict):
244             return base_preset == cur_preset
245         keys = set(base_preset) | set(cur_preset)
246         return all(base_preset.get(k) == cur_preset.get(k) for k in keys if k != "name")
247
248
249     def compare_config(name: str, base_sum: Dict[str, Any], cur_sum: Dict[str, Any],
250                       tols: Tolerances) -> ConfigComparison:
251         """Compare one config's current summary to its baseline summary.
252
253         * If the preset definition or the net-crossings gate changed, the whole config is
STALE
254         (numbers aren't comparable) ? flag it, skip gating.
255         * If the corpus (video set) changed, the aggregate is not comparable (different
videos) ?
256         skip the aggregate gate + flag added/removed, but STILL run the per-video gate on
the
257         INTERSECTION (a code regression on a shared video must still trip the wire).
258         """
259         cmp = ConfigComparison(config=name)
260
261         # 1) Config-definition drift ? stale, not gateable.
262         if not _preset_comparable(base_sum.get("preset"), cur_sum.get("preset")) \
or base_sum.get("min_crossings") != cur_sum.get("min_crossings"):
263

```

```

264         cmp.status = "stale"
265         cmp.aggregate_gated = False
266         cmp.findings.append(Finding(config=name, scope="config", metric="preset",
kind="stale"))
267         return cmp
268
269         base_pv = base_sum.get("per_video", {})
270         cur_pv = cur_sum.get("per_video", {})
271         base_names, cur_names = set(base_pv), set(cur_pv)
272         cmp.added_videos = sorted(cur_names - base_names)
273         cmp.removed_videos = sorted(base_names - cur_names)
274         corpus_changed = bool(cmp.added_videos or cmp.removed_videos)
275
276         # 2) Aggregate gate (only when the corpus matches ? means over different sets aren't
comparable).
277         if corpus_changed:
278             cmp.aggregate_gated = False
279             cmp.status = "stale"
280             cmp.findings.append(Finding(config=name, scope="config", metric="corpus",
kind="stale"))
281         else:
282             base_agg = base_sum.get("aggregate", {})
283             cur_agg = cur_sum.get("aggregate", {})
284             for m in GATED_HIGHER:
285                 b, c = base_agg.get(m), cur_agg.get(m)
286                 if b is None or c is None:
287                     continue
288                 if c < b - tols.metric_tol:
289                     cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
290                 elif c > b + tols.metric_tol:
291                     cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
292             for m in GATED_LOWER:
293                 b, c = base_agg.get(m), cur_agg.get(m)
294                 if b is None or c is None:
295                     continue
296                 if c > b + tols.rate_tol:
297                     cmp.findings.append(Finding(name, "aggregate", m, "regression", b, c))
298                 elif c < b - tols.rate_tol:
299                     cmp.findings.append(Finding(name, "aggregate", m, "improvement", b, c))
300
301         # 3) Per-video over-seg guard on the intersection (always ? catches per-video code
regressions).
302         for vid in sorted(base_names & cur_names):
303             for m in PER_VIDEO_GUARD:
304                 b, c = base_pv[vid].get(m), cur_pv[vid].get(m)
305                 if b is None or c is None:
306                     continue
307                 if c > b + tols.over_seg_tol:
308                     cmp.findings.append(Finding(name, "per_video", m, "regression",
float(b), float(c), video=vid))
309
310         if cmp.regressions:
311             cmp.status = "regression"
312         elif cmp.status != "stale":
313             cmp.status = "ok"
314         return cmp
315
316
317     @dataclass
318     class NightlyResult:
319         comparisons: List[ConfigComparison] = field(default_factory=list)
320         skipped: Dict[str, List[str]] = field(default_factory=dict) # config -> manifest
videos not scored
321
322         @property

```

```

323     def has_regression(self) -> bool:
324         return any(c.status == "regression" for c in self.comparisons)
325
326     @property
327     def has_stale(self) -> bool:
328         return any(c.status == "stale" for c in self.comparisons)
329
330     def overall_status(self) -> str:
331         if self.has_regression:
332             return "REGRESSION"
333         if self.has_stale:
334             return "STALE"
335         return "PASS"
336
337     def exit_code(self) -> int:
338         if self.has_regression:
339             return 1
340         if self.has_stale:
341             return 4
342         return 0
343
344
345     def compare_all(baseline: Dict[str, Any], current: Dict[str, Any],
346                    tols: Tolerances, skipped: Optional[Dict[str, List[str]]] = None) ->
NightlyResult:
347         """Compare every tracked config's current summary to the baseline summary."""
348         res = NightlyResult(skipped=skipped or {})
349         base_cfgs = baseline.get("configs", {})
350         cur_cfgs = current.get("configs", {})
351         for name in cur_cfgs:
352             if name not in base_cfgs:
353                 cmp = ConfigComparison(config=name, status="stale", aggregate_gated=False)
354                 cmp.findings.append(Finding(config=name, scope="config",
metric="missing_in_baseline", kind="stale"))
355                 res.comparisons.append(cmp)
356                 continue
357                 res.comparisons.append(compare_config(name, base_cfgs[name], cur_cfgs[name],
tols))
358         return res
359
360
361     # ----- #
362     # Report formatting (pure).
363     # ----- #
364     def _fmt(v: Optional[float]) -> str:
365         return "?" if v is None else f"{v:.4f}"
366
367
368     def format_report(baseline: Dict[str, Any], current: Dict[str, Any],
369                      result: NightlyResult, tols: Tolerances, date: str) -> str:
370         L: List[str] = []
371         status = result.overall_status()
372         badge = {"PASS": "? PASS", "REGRESSION": "? REGRESSION", "STALE": "? STALE (baseline
refresh needed)"}[status]
373         L.append(f"# Nightly rally-quality regression ? {date}")
374         L.append("")
375         L.append(f"**Status: {badge}** · exit code {result.exit_code()}")
376         L.append("")
377         L.append(f"- HEAD: `{current.get('git_commit', '?')}` · baseline: "
378                 f"`{baseline.get('git_commit', '?')}` (snapshot
{baseline.get('generated_at', '?')})")
379         L.append(f"- Corpus manifest: `{current.get('manifest', '?')}` ·
IoU={current.get('iou', DEFAULT_IoU)}")
380         L.append(f"- Tolerances: metric ±{tols.metric_tol}, rate ±{tols.rate_tol}, "
381                 f"per-video over-seg ±{tols.over_seg_tol}")

```



```

382     L.append("")
383
384     # Up-front regression / stale summary.
385     regs = [f for c in result.comparisons for f in c.regressions]
386     if regs:
387         L.append("## ? Regressions")
388         L.append("")
389         for f in regs:
390             L.append(f"- {f.message()}")
391         L.append("")
392     if result.has_stale:
393         L.append("## ? Stale / not comparable")
394         L.append("")
395         for c in result.comparisons:
396             if c.status == "stale":
397                 if c.added_videos:
398                     L.append(f"- `{c.config}`: corpus added {c.added_videos}")
399                 if c.removed_videos:
400                     L.append(f"- `{c.config}`: corpus removed {c.removed_videos}")
401                 if not c.added_videos and not c.removed_videos:
402                     L.append(f"- `{c.config}`: config definition changed vs baseline ?
re-snapshot with --update-baseline")
403                 L.append("- Action: `python scripts/nightly_regression.py --update-baseline`
(after confirming the change is intended).")
404                 L.append("")
405
406         # Skipped (no silent caps).
407         skipped_any = {k: v for k, v in result.skipped.items() if v}
408         if skipped_any:
409             L.append("## ? Skipped manifest videos (missing trajectory or GT ? NOT silently
dropped)")
410             L.append("")
411             for cfg, vids in skipped_any.items():
412                 L.append(f"- `{cfg}`: {vids}")
413             L.append("")
414
415         # Per-config detail.
416         cur_cfgs = current.get("configs", {})
417         base_cfgs = baseline.get("configs", {})
418         for cmp in result.comparisons:
419             cur = cur_cfgs.get(cmp.config, {})
420             base = base_cfgs.get(cmp.config, {})
421             L.append(f"## Config `{cmp.config}` ? {cur.get('note', '')}")
422             L.append("")
423             L.append(f"_status: {cmp.status}; n={cur.get('n', '?')} (baseline
n={base.get('n', '?')})")
424             L.append("")
425             cagg = cur.get("aggregate", {})
426             bagg = base.get("aggregate", {})
427             L.append("| metric | baseline | current | ? | gated |")
428             L.append("|---|---|---|---|---|")
429             for m in AGG_SNAPSHOT:
430                 b, c = bagg.get(m), cagg.get(m)
431                 d = (c - b) if (b is not None and c is not None) else None
432                 gated = "?" if (m in GATED_HIGHER or m in GATED_LOWER) and
cmp.aggregate_gated else ""
433                 L.append(f"| {m} | {fmt(b)} | {fmt(c)} | {fmt(d)} | {gated} |")
434             L.append("")
435             # Per-video over-seg guard table.
436             cpv = cur.get("per_video", {})
437             bpv = base.get("per_video", {})
438             names = sorted(set(cpv) | set(bpv))
439             if names:
440                 L.append("| video | tolF1 (b?c) | split (b?c) | merge (b?c) |")
441                 L.append("|---|---|---|---|")

```

```

442         for v in names:
443             cb, cc = bpv.get(v, {}), cpv.get(v, {})
444             tf = f"{{_fmt(cb.get('tol_f1'))}}?{{_fmt(cc.get('tol_f1'))}}"
445             sp = f"{{cb.get('split_count', '?')}}?{{cc.get('split_count', '?')}}"
446             mg = f"{{cb.get('merge_count', '?')}}?{{cc.get('merge_count', '?')}}"
447             flag = " ?" if any(f.video == v and f.kind == "regression" for f in
cmp.findings) else ""
448             L.append(f"| {v}{{flag}} | {{tf}} | {{sp}} | {{mg}} |")
449             L.append("")
450             imps = cmp.improvements
451             if imps:
452                 L.append("_Improvements over baseline (consider `--update-baseline` to
ratchet the floor up):_")
453                 for f in imps:
454                     L.append(f"- {f.message()}")
455                 L.append("")
456
457             L.append("---")
458             L.append("_Tripwire only ? re-scores CACHED trajectories (no GPU/Gemini/WASB). Does
not change "
459                 "the promotion gate. See docs/R2_EVAL_METRIC_DESIGN.md + memory/eval-dual-
set-regression._")
460             return "\n".join(L)
461
462
463     # ----- #
464     # I/O shell.
465     # ----- #
466     def _git_commit() -> str:
467         import subprocess
468         try:
469             out = subprocess.run(["git", "-C", REPO_ROOT, "rev-parse", "--short", "HEAD"],
capture_output=True, text=True, timeout=10)
470             return out.stdout.strip() or "?"
471         except Exception:
472             return "?"
473
474
475
476     def _display_path(path: str) -> str:
477         """A clean, portable form for the committed baseline: ``output/<file>`` when the
path lives
478         in an ``output`` dir (the production layout), else repo-relative, else the path as
given.
479         Metadata only ? the comparison never reads it."""
480         norm = path.replace("\\", "/")
481         parent = os.path.basename(os.path.dirname(norm))
482         if parent == "output":
483             return "output/" + os.path.basename(norm)
484         try:
485             rel = os.path.relpath(path, REPO_ROOT)
486             if not rel.startswith(".."):
487                 return rel.replace("\\", "/")
488         except ValueError:
489             pass
490         return norm
491
492
493     def run_corpus(configs: List[NightlyConfig], manifest: str, features_dir: str,
494                   iou: float = DEFAULT_IOU) -> Tuple[Dict[str, Any], Dict[str, List[str]]]:
495         """Score every tracked config over the golden corpus. Returns (snapshot, skipped-
per-config).
496
497         ``snapshot`` is the serialisable structure stored as / compared to the baseline.
498         ``skipped``
499         lists manifest videos NOT scored (missing trajectory or empty GT) per config ?

```

```

surfaced in
499     the report so the corpus can't silently shrink.""
500     from backend.eval import rally_seg_eval
501
502     golden = rally_seg_eval.load_golden(manifest, features_dir)
503     manifest_names = [v["name"] for v in golden]
504     if not golden:
505         raise RuntimeError(f"no golden videos loaded from {manifest}")
506
507     snapshot: Dict[str, Any] = {"manifest": _display_path(manifest), "iou": iou,
"configs": {}}
508     skipped: Dict[str, List[str]] = {}
509     scored_any = False
510     for cfg in configs:
511         result = rally_seg_eval.evaluate(golden, cfg.preset, iou=iou,
min_crossings=cfg.min_crossings)
512         summary = summarize_run(result)
513         summary["note"] = cfg.note
514         snapshot["configs"][cfg.name] = summary
515         scored = set(summary["per_video"])
516         skipped[cfg.name] = [n for n in manifest_names if n not in scored]
517         scored_any = scored_any or bool(scored)
518     if not scored_any:
519         raise RuntimeError(
520             f"scored 0 videos for every config ? are the trajectory CSVs in {manifest}
present "
521             f"and do the {features_dir}/<name>_golden_features.json files have 'gts'?")
522     return snapshot, skipped
523
524
525     def load_baseline(path: str) -> Optional[Dict[str, Any]]:
526         if not os.path.exists(path):
527             return None
528         with open(path, encoding="utf-8") as fh:
529             return json.load(fh)
530
531
532     def save_baseline(path: str, snapshot: Dict[str, Any], tols: Tolerances) -> None:
533         out = dict(snapshot)
534         out["generated_at"] = _dt.date.today().isoformat()
535         out["git_commit"] = _git_commit()
536         out["tolerances"] = tols.to_dict()
537         out["_doc"] = ("Frozen nightly rally-quality regression baseline. Regenerate with "
538             "'python scripts/nightly_regression.py --update-baseline` after an
intended "
539             "'improvement or a corpus change. Tied to the LOCAL golden corpus (not
committed).")
540         os.makedirs(os.path.dirname(path), exist_ok=True)
541         tmp = path + ".tmp"
542         with open(tmp, "w", encoding="utf-8") as fh:
543             json.dump(out, fh, indent=2, sort_keys=True)
544             fh.write("\n")
545         os.replace(tmp, path)
546
547
548     def build_parser() -> argparse.ArgumentParser:
549         p = argparse.ArgumentParser(description="Nightly rally-quality regression tripwire
(offline, CPU).")
550         p.add_argument("--manifest", default=DEFAULT_MANIFEST, help="golden manifest JSON")
551         p.add_argument("--features-dir", default=DEFAULT_FEATURES_DIR,
552             help="dir holding <name>_golden_features.json")
553         p.add_argument("--baseline", default=DEFAULT_BASELINE, help="frozen baseline JSON
path")
554         p.add_argument("--report-dir", default=DEFAULT_REPORT_DIR, help="where dated reports
are written")

```

```

555     p.add_argument("--iou", type=float, default=DEFAULT_IOU)
556     p.add_argument("--metric-tol", type=float, default=DEFAULT_METRIC_TOL,
557                     help="F1-like aggregate drop that counts as a regression")
558     p.add_argument("--rate-tol", type=float, default=DEFAULT_RATE_TOL,
559                     help="merge_rate rise that counts as a regression")
560     p.add_argument("--over-seg-tol", type=int, default=DEFAULT_OVER_SEG_TOL,
561                     help="per-video split/merge-count increase allowed before it's a
regression")
562     p.add_argument("--update-baseline", action="store_true",
563                     help="snapshot the current numbers as the new baseline instead of
checking")
564     p.add_argument("--date", default=None, help="override the report date (YYYY-MM-DD;
for tests)")
565     p.add_argument("--no-report", action="store_true", help="skip writing the dated
report file")
566     return p
567
568
569     def main(argv: Optional[List[str]] = None) -> int:
570         import logging
571         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
572         log = logging.getLogger("nightly_regression")
573         args = build_parser().parse_args(argv)
574         tols = Tolerances(metric_tol=args.metric_tol, rate_tol=args.rate_tol,
over_seg_tol=args.over_seg_tol)
575         date = args.date or _dt.date.today().isoformat()
576
577         if not os.path.exists(args.manifest):
578             log.error("manifest not found: %s ? the golden corpus must be present locally.",
args.manifest)
579             return 2
580
581         try:
582             snapshot, skipped = run_corpus(tracked_configs(), args.manifest,
args.features_dir, args.iou)
583         except RuntimeError as e:
584             log.error("%s", e)
585             return 2
586         snapshot["git_commit"] = _git_commit()
587         snapshot["generated_at"] = date
588
589         if args.update_baseline:
590             save_baseline(args.baseline, snapshot, tols)
591             log.info("baseline snapshot written to %s", args.baseline)
592             for cfg, s in snapshot["configs"].items():
593                 agg = s["aggregate"]
594                 log.info(" %s: n=%s tolF1=%.3f f1@0.5=%.3f merge_rate=%.3f",
595                         cfg, s["n"], agg.get("tol_f1") or 0.0, agg.get("f1@0.5") or 0.0,
596                         agg.get("merge_rate") or 0.0)
597             return 0
598
599         baseline = load_baseline(args.baseline)
600         if baseline is None:
601             log.error("no baseline at %s ? snapshot one with --update-baseline (then commit
it).", args.baseline)
602             return 3
603
604         result = compare_all(baseline, snapshot, tols, skipped)
605         report = format_report(baseline, snapshot, result, tols, date)
606
607         if not args.no_report:
608             os.makedirs(args.report_dir, exist_ok=True)
609             md_path = os.path.join(args.report_dir, f"{date}.md")
610             json_path = os.path.join(args.report_dir, f"{date}.json")
611             with open(md_path, "w", encoding="utf-8") as fh:

```

```

612         fh.write(report + "\n")
613     with open(json_path, "w", encoding="utf-8") as fh:
614         json.dump({"date": date, "status": result.overall_status(),
615                  "exit_code": result.exit_code(), "current": snapshot,
616                  "regressions": [f.message() for c in result.comparisons for f in
c.regressions]}},
617                  fh, indent=2, sort_keys=True)
618     log.info("report written to %s", md_path)
619
620     print(report)
621     code = result.exit_code()
622     if code == 1:
623         log.error("NIGHTLY REGRESSION DETECTED (%d) ? see %s",
624                  sum(len(c.regressions) for c in result.comparisons), args.report_dir)
625     elif code == 4:
626         log.warning("baseline STALE ? corpus/config changed; refresh with --update-
baseline")
627     else:
628         log.info("nightly regression check PASSED")
629     return code
630
631
632 if __name__ == "__main__":
633     raise SystemExit(main())
634

```

6. user (2026-06-18T07:47:57.026Z)

```

1     import sys
2     import os
3     import subprocess
4     import shutil
5
6     # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7     # root (requirements.txt, config.json). Resolve the root from this file's
8     # location so it works regardless of the caller's cwd.
9     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11     def check_command(cmd):
12         """Check if a system command is available."""
13         return shutil.which(cmd) is not None
14
15     def run_pip_install():
16         """Install required Python packages."""
17         print("Installing Python packages from requirements.txt...")
18
19         # Check if NVIDIA GPU is present
20         has_gpu = check_command("nvidia-smi")
21
22         requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23         pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25         if has_gpu:
26             print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
27             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
28         else:
29             print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
30             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
31
32         try:
33             subprocess.check_call(pip_cmd)
34             print("[SUCCESS] Python packages installed.")
35             return True

```

```

36         except subprocess.CalledProcessError as e:
37             print(f"[ERROR] Failed to install Python packages: {e}")
38             return False
39
40     def init_default_config():
41         """Initialize a default config.json if it doesn't exist.
42
43         Now delegates to the canonical backend.config loader so that defaults
44         stay in sync with the Pydantic models (single source of truth).
45         """
46         try:
47             from backend.config import save_default_config
48         except Exception as e:
49             print(f"[ERROR] Could not import backend.config loader: {e}")
50             print("[INFO] Falling back to no-op for config initialization.")
51             return
52
53         config_path = os.path.join(REPO_ROOT, "config.json")
54         if os.path.exists(config_path):
55             print(f"[INFO] {config_path} already exists. Skipping initialization.")
56             return
57
58         try:
59             saved_path = save_default_config(config_path)
60             print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
61         except Exception as e:
62             print(f"[ERROR] Failed to create config.json via loader: {e}")
63
64     def run_diagnostics():
65         """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
66         print("=" * 50)
67         print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
68         print("=" * 50)
69
70         # 1. Check Python Version
71         print(f"Python Version: {sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
72         if sys.version_info.major < 3 or (sys.version_info.major == 3 and sys.version_info.minor < 8):
73             print("[WARNING] Python 3.8+ is highly recommended!")
74         else:
75             print("[PASS] Python version check passed.")
76
77         print("-" * 50)
78
79         # 2. Check FFmpeg / FFprobe
80         ffmpeg_ok = check_command("ffmpeg")
81         ffprobe_ok = check_command("ffprobe")
82
83         if ffmpeg_ok:
84             print("[PASS] FFmpeg is installed and available in PATH.")
85         else:
86             print("[FAIL] FFmpeg is NOT found in your PATH.")
87
88         if ffprobe_ok:
89             print("[PASS] FFprobe is installed and available in PATH.")
90         else:
91             print("[FAIL] FFprobe is NOT found in your PATH.")
92
93         if not (ffmpeg_ok and ffprobe_ok):
94             print("\n" + "!" * 50)
95             print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
96             print("How to install FFmpeg on Windows:")
97             print("  Option 1: Open PowerShell as Administrator and run:")

```

```

98         print("            winget install Gyan.FFmpeg")
99         print(" Option 2: Download the build from https://ffmpeg.org/download.html")
100        print("            and add its 'bin' directory to your System PATH environment
variable.")
101        print("!" * 50 + "\n")
102    else:
103        print("[SUCCESS] All essential media command line tools are ready.")
104
105    print("-" * 50)
106
107    # 3. Check ML / GPU (Optional but good)
108    try:
109        import torch
110        cuda_ok = torch.cuda.is_available()
111        if cuda_ok:
112            print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
113        else:
114            print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
115    except ImportError:
116        print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
117
118    print("=" * 50)
119    return ffmpeg_ok and ffprobe_ok
120
121    if __name__ == "__main__":
122        init_default_config()
123        media_ok = run_diagnostics()
124        run_pip_install()
125        print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
126

```

7. assistant (2026-06-18T07:47:59.720Z)

Now let me read the remaining in-scope files: the reporting, tools, and the other scripts.

8. user (2026-06-18T07:48:00.734Z)

```

1    """Build an observability report for one indexer run.
2
3    Pure-ish harvest: given the validation results/context, the persisted segments,
4    and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5    *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6    so this module only records facts faithfully ? it does not decide warnings.
7
8    Imported only after `backend.reporting` confirms obsreport is installed, so it may
9    import obsreport freely.
10   """
11   from __future__ import annotations
12
13   import os
14   from pathlib import Path
15   from typing import Any, Dict, List, Optional
16
17   try:
18       import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
19   except ImportError:
20       obsreport = None # type: ignore[assignment]
21
22   _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23   _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"
24
25

```

```

26     def load_spec() -> "obsreport.ReportSpec":
27         return obsreport.load_spec(_SPEC_PATH)
28
29
30     # --- media metadata -----
31     def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32         """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33         distinguishes a real probe from the absence of one (the fallback footgun)."""
34         meta: Dict[str, Any] = {"fps_source": "unknown"}
35         if video_path and os.path.exists(video_path):
36             try:
37                 meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38             except OSError:
39                 pass
40
41         ffmata = (val_context or {}).get("ffprobe_meta") or {}
42         streams = ffmata.get("streams") or []
43         fmt = ffmata.get("format") or {}
44         if streams:
45             s = streams[0]
46             w, h = s.get("width"), s.get("height")
47             meta["width"], meta["height"] = w, h
48             if w and h:
49                 meta["resolution"] = f"{w}x{h}"
50             fps_str = s.get("r_frame_rate", "0/1")
51             try:
52                 num, den = (int(x) for x in fps_str.split("/"))
53                 if den:
54                     meta["fps"] = round(num / den, 2)
55                     meta["fps_source"] = "probed"
56             except (ValueError, ZeroDivisionError):
57                 pass
58             if s.get("codec_name"):
59                 meta["container"] = fmt.get("format_name") or s.get("codec_name")
60         if fmt.get("duration"):
61             try:
62                 meta["duration_s"] = round(float(fmt["duration"]), 1)
63             except (ValueError, TypeError):
64                 pass
65         if fmt.get("format_name"):
66             meta["container"] = fmt.get("format_name")
67
68         # Audio readiness signal (for Input Quality UX extension per review item 2).
69         # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70         # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71         # We only record presence here (cheap, from ffprobe already in context); the actual
72         # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73         # tool for now.
74         audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75         meta["audio_present"] = len(audio_streams) > 0
76         return meta
77
78
79     # --- stage builders -----
80     def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81         with rec.stage("validation") as st:
82             total = len(val_results)
83             passed = sum(1 for r in val_results if getattr(r, "passed", False))
84             st.add_metric("checks_passed", passed)
85             st.add_metric("checks_total", total)
86             failures = [r for r in val_results if not getattr(r, "passed", True)]
87             for r in failures:
88                 st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,

```



```

'message', '')})")
89         st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92     def _segmentation_stage(
93         rec: "obsreport.RunRecorder",
94         play_segments: List[Dict[str, Any]],
95         candidates: List[Dict[str, Any]],
96         segmenter_requested: Optional[str],
97         segmenter_actual: Optional[str],
98         config_default: Optional[str],
99         was_reingest: bool,
100         ran: bool,
101     ) -> None:
102         with rec.stage("segmentation") as st:
103             if not ran:
104                 st.status = "not_run"
105             seg_count = len(play_segments)
106             total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107             detector_tag = None
108             if play_segments:
109                 detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110             st.add_metric("candidate_windows", len(candidates))
111             st.add_metric("segment_count", seg_count, audience="both")
112             st.add_metric("total_play_s", total_play, unit="s")
113             # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114             # pick it, and it differs from config's default (the genuine surprise case).
115             # An explicit --segmenter override or a halted run is not a divergence.
116             matches_default = True
117             if ran and segmenter_actual and config_default:
118                 matches_default = (segmenter_actual == config_default)
119             st.details.update({
120                 "segmenter_requested": segmenter_requested,
121                 "segmenter_actual": segmenter_actual,
122                 "config_default": config_default,
123                 "segmenter_explicitly_requested": segmenter_requested is not None,
124                 "matches_config_default": matches_default,
125                 "was_reingest": was_reingest,
126                 "detector": detector_tag,
127             })
128             # Surface metadata trustworthiness at the data level (not just in a warning):
129             # the motion baseline fabricates per-segment metadata; everything else measures
130             it.
131             if ran and segmenter_actual:
132                 st.details["metadata_source"] = (
133                     "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134                     else "measured (detector/AI)"
135                 )
136
137     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                 play_segments: List[Dict[str, Any]], ran: bool,
139                 run_signals: Optional[Dict[str, Any]] = None) -> None:
140         """Only meaningful for AI-based segmenters. Records whether the provider key was
141         present ? the signal the silent_provider_noop rule keys on ? plus any in-process
142         run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
143         ai_based = segmenter_actual in _AI_SEGMENTERS
144         provider = (config.get("ai_provider") or "gemini")
145         key_env = f"{provider.upper()}_API_KEY"
146         with rec.stage("ai_handoff") as st:
147             if not ran or not ai_based:
148                 st.status = "not_run" if not ran else "skipped"
149             st.details.update({
150                 "ai_based": ai_based,

```

```

151         "provider": provider if ai_based else None,
152         "model": config.get("ai_model") if ai_based else None,
153         "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
154         # Chunks the provider refused as over its upload cap (no analysis ran for
155         # those time ranges). Detail name couples to the oversize_clip_skipped rule.
156         "oversize_clips_skipped": int((run_signals or
157 {}).get("oversize_clips_skipped", 0)) if ai_based else None,
158     })
159     if ai_based:
160         st.add_metric("confirmed_segments", len(play_segments))
161
162 def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
163               video_meta: Dict[str, Any]) -> None:
164     total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
165     coverage = None
166     dur = video_meta.get("duration_s")
167     if dur:
168         coverage = round(100.0 * total / dur, 1)
169     notes = []
170     if not play_segments:
171         notes.append("No highlights were produced for this video.")
172     rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
173                       coverage_pct=coverage, notes=notes)
174
175 # --- top-level -----
176 def record_run(
177     *,
178     video_id: str,
179     video_path: Optional[str],
180     config: Dict[str, Any],
181     val_results: List[Any],
182     val_context: Optional[Dict[str, Any]],
183     play_segments: List[Dict[str, Any]],
184     candidates: List[Dict[str, Any]],
185     segmenter_requested: Optional[str] = None,
186     segmenter_actual: Optional[str] = None,
187     was_reingest: bool = False,
188     segmentation_ran: bool = True,
189     tool_version: Optional[str] = None,
190     spec: Optional["obsreport.ReportSpec"] = None,
191     run_signals: Optional[Dict[str, Any]] = None,
192     **recorder_kwargs: Any,
193 ) -> "obsreport.RunRecorder":
194     """Assemble a finalized recorder for an indexer run (does NOT write)."""
195     rec = obsreport.RunRecorder(
196         "badminton-highlight-indexer",
197         tool_version=tool_version,
198         video_id=video_id,
199         video_id_kind="file_md5",
200         video_path=video_path,
201         spec=spec if spec is not None else load_spec(),
202         **recorder_kwargs,
203     )
204     vm = video_meta_from_context(val_context, video_path)
205     # If we never probed an fps but segmentation still ran, the segmenter must have
206     # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
207     # AND show the assumed numeric value so the report reads "30 (fallback)" not "?
208     # (fallback)".
209     if vm.get("fps_source") == "unknown" and segmentation_ran:
210         vm["fps_source"] = "fallback"
211         if vm.get("fps") is None:
212             vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
213 / 1280)

```

```

213         rec.set_video_meta(**vm)
214
215         _validation_stage(rec, val_results)
216         config_default = (config.get("indexing") or {}).get("default_segmenter")
217         _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
218                             segmenter_actual, config_default, was_reingest,
segmentation_ran)
219         _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
run_signals)
220         _highlights(rec, play_segments, vm)
221
222         rec.finalize()
223         return rec
224

```

9. user (2026-06-18T07:48:01.167Z)

```

1     """In-process signal channel: pipeline internals -> the per-video report.
2
3     Some report-worthy facts (e.g. the provider refusing an oversize chunk) surface
4     deep inside a segmenter's worker threads, with no return path to the harvest ?
5     segmenters return only their segment list. Layers record those facts here keyed
6     by video_id; `emit_indexer_report` pops them when the run's report is built.
7
8     Process-local and best-effort BY DESIGN: recording must never break a run, the
9     store is consumed (popped) once per report, and signals for a video are simply
10    dropped if no report is emitted. No obsreport dependency ? importable everywhere.
11    """
12    from __future__ import annotations
13
14    import threading
15    from typing import Any, Dict
16
17    _lock = threading.Lock()
18    _signals: Dict[str, Dict[str, Any]] = {}
19
20
21    def incr(video_id: str, key: str, by: int = 1) -> None:
22        """Add `by` to the counter `key` for this video's in-flight run."""
23        if not video_id or by == 0:
24            return
25        with _lock:
26            sig = _signals.setdefault(video_id, {})
27            sig[key] = int(sig.get(key, 0)) + by
28
29
30    def pop(video_id: str) -> Dict[str, Any]:
31        """Consume (remove and return) all signals recorded for this video's run."""
32        with _lock:
33            return _signals.pop(video_id, {})
34

```

10. assistant (2026-06-18T07:48:03.687Z)

Now the tools and remaining scripts.

11. user (2026-06-18T07:48:04.649Z)

```

1     """Minimal Value Layer exports (first slice of review item 3).
2
3     Provides timeline exports (EDL, JSON, CSV) and basic stats over the
4     play_segments returned by the indexer (or query_play_segments). These are
5     the "time-saved editing is table stakes" deliverables and also give a
6     machine-readable foundation for later human correction / golden feeding

```

```

7     and clip galleries.
8
9     Kept deliberately small and pure (no DB, no side effects) so they are
10    easy to call from CLI, notebooks, or a future API endpoint.
11    """
12
13    from __future__ import annotations
14
15    import json
16    from typing import Any, Dict, Iterable, List, Optional
17
18
19    def _to_list(segments: Iterable[Dict[str, Any]]) -> List[Dict[str, Any]]:
20        return list(segments) if not isinstance(segments, list) else segments
21
22
23    def _segment_metadata(s: Dict[str, Any]) -> Dict[str, Any]:
24        """A play segment's metadata as a dict, whether stored inline (segmenter output)
25        or as a JSON string (DB round-trip via query_play_segments)."""
26        md = s.get("metadata") or {}
27        if isinstance(md, str):
28            try:
29                md = json.loads(md)
30            except Exception:
31                md = {}
32        return md if isinstance(md, dict) else {}
33
34
35    def _fmt_hms(seconds: float) -> str:
36        """HH:MM:SS, matching the segmenters' _fmt_ts so the summary block lines up with
37        the per-rally [rally] log lines and the existing timestamp style."""
38        total = max(0, int(float(seconds)))
39        h, rem = divmod(total, 3600)
40        m, s = divmod(rem, 60)
41        return f"{h:02d}:{m:02d}:{s:02d}"
42
43
44    def format_rally_summary(
45        segments: Iterable[Dict[str, Any]], title: str = "Rally summary"
46    ) -> str:
47        """Render a compact, scannable end-of-run block for manual QA against the rally CSV.
48
49        One header line (total rallies + total highlight duration in HH:MM:SS) followed by a
50        per-rally table ``# start end dur(s) shot_count`` in time order. Pure (no DB, no
51        I/O):
52
53        the same play_segment dicts that feed ``segments_to_csv`` / the rally CSV go in and
54        text comes out, so the log and CSV line up row-for-row for eyeball verification.
55
56        A segment with no ``shot_count`` in its metadata renders the count as ``-``.
57        """
58        segs = _to_list(segments)
59        total_dur = sum(
60            float(s.get("duration", 0)) - float(s.get("start_time", 0))
61            for s in segs
62        )
63        header = (
64            f"=== {title}: {len(segs)} rallies, "
65            f"{_fmt_hms(total_dur)} ({total_dur:.1f}s) total play ==="
66        )
67        if not segs:
68            return header + "\n (no rallies detected)"
69
70        lines = [header, f"{'#':>3} {'start':<8} {'end':<8} {'dur(s)':>7} {'shots':>6}"]
71        for i, s in enumerate(segs, start=1):

```

```

70         start = float(s.get("start_time", 0.0))
71         end = float(s.get("end_time", start))
72         dur = float(s.get("duration", end - start))
73         shot = _segment_metadata(s).get("shot_count")
74         shot_str = "-" if shot is None else str(shot)
75         lines.append(
76             f"{i:>3}  {_fmt_hms(start):<8}  {_fmt_hms(end):<8}  {dur:>7.1f}
{shot_str:>6}"
77         )
78         return "\n".join(lines)
79
80
81     def basic_stats(segments: Iterable[Dict[str, Any]]) -> Dict[str, Any]:
82         """Return simple aggregate stats for a list of play segments.
83
84         Keys: count, total_duration_s, avg_duration_s, min_duration_s, max_duration_s.
85         All durations are in seconds (float).
86         """
87         segs = _to_list(segments)
88         if not segs:
89             return {
90                 "count": 0,
91                 "total_duration_s": 0.0,
92                 "avg_duration_s": 0.0,
93                 "min_duration_s": 0.0,
94                 "max_duration_s": 0.0,
95             }
96         durs = [float(s.get("duration", (s.get("end_time", 0) - s.get("start_time", 0))))
97 for s in segs]
98         total = sum(durs)
99         return {
100             "count": len(segs),
101             "total_duration_s": round(total, 2),
102             "avg_duration_s": round(total / len(durs), 2),
103             "min_duration_s": round(min(durs), 2),
104             "max_duration_s": round(max(durs), 2),
105         }
106
107     def segments_to_json(
108         segments: Iterable[Dict[str, Any]],
109         video_id: str,
110         video_path: Optional[str] = None,
111         meta: Optional[Dict[str, Any]] = None,
112     ) -> Dict[str, Any]:
113         """Return a JSON-serializable timeline dict.
114
115         Shape (stable):
116         {
117             "video_id": "...",
118             "video_path": "...",
119             "meta": {...},
120             "segments": [ {"id":, "start_time":, "end_time":, "duration":, "metadata": {...}},
... ],
121             "stats": basic_stats(...)
122         }
123         """
124         segs = _to_list(segments)
125         return {
126             "video_id": video_id,
127             "video_path": video_path,
128             "meta": meta or {},
129             "segments": segs,
130             "stats": basic_stats(segs),
131         }

```

```

132
133
134 def segments_to_csv(segments: Iterable[Dict[str, Any]], include_header: bool = True) ->
str:
135     """Return a CSV string (start,end,duration,shot_count,detector,...) .
136
137     Compatible with many editors and the existing golden/slicer schemas
138     (extra columns are harmless).
139     """
140     segs = _to_list(segments)
141     lines: List[str] = []
142     if include_header:
143         lines.append("start_time,end_time,duration,shot_count,shot_count_confidence,detector,id")
144     for s in segs:
145         md = s.get("metadata") or {}
146         shot = md.get("shot_count", "")
147         conf = md.get("shot_count_confidence", "")
148         det = md.get("detector", "")
149         lines.append(
150             f"{s.get('start_time',0)},{s.get('end_time',0)},{s.get('duration',0)},{s.get('shot_count',0)},{s.get('shot_count_confidence',0)},{s.get('detector',0)},{s.get('id','')}"
151         )
152     return "\n".join(lines) + "\n"
153
154
155
156 def segments_to_edl(segments: Iterable[Dict[str, Any]], title: str = "Highlights") ->
str:
157     """Return a simple EDL (CMX 3600-ish) for the segments.
158
159     Many NLEs will ingest this for a rough cut. Uses record timecode 00:00:00:00
160     as origin; clips are placed sequentially with the original source in/out.
161     """
162     segs = _to_list(segments)
163     lines: List[str] = [f"TITLE: {title}", "FCM: NON-DROP FRAME"]
164     for i, s in enumerate(segs, 1):
165         start = float(s.get("start_time", 0.0))
166         end = float(s.get("end_time", start))
167         # EDL is HH:MM:SS:FF (assume 30fps for the timecode display; real edit rate
168         # is irrelevant for a rough cut ? the numbers are just labels here).
169         def _tc(t: float) -> str:
170             h = int(t // 3600)
171             m = int((t % 3600) // 60)
172             s_ = int(t % 60)
173             f = int((t - int(t)) * 30) # 30 fps display
174             return f"{h:02d}:{m:02d}:{s_:02d}:{f:02d}"
175         rec_in = _tc((i - 1) * 0.0) # place back-to-back in the EDL timeline
176         rec_out = _tc((i - 1) * 0.0 + (end - start))
177         src_in = _tc(start)
178         src_out = _tc(end)
179         lines.append(f"{i:03d} AX V C {rec_in} {rec_out} {src_in} {src_out}")
180     lines.append(f"* FROM CLIP NAME: {title} {i}")
181     lines.append("")
182     return "\n".join(lines)
183

```

12. user (2026-06-18T07:48:05.106Z)

```

1     """Slice individual rally clips out of a (long) annotated video.
2
3     Use cases:
4     - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5       boundaries against the labels.
6     - **Per-video calibration/training** ? grab a handful of labeled rallies as the

```

```

7     supervision the segmenter is tuned/adapted on for that specific video.
8
9     Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10    end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11        <out_dir>/<video_stem>__rally_<n>.mp4
12
13    Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14    shows the same lead-in/out the model is trained and evaluated against. The window
15    math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in
16    `slice_rallies`.
17    """
18    from __future__ import annotations
19
20    import argparse
21    import csv
22    import os
23    import subprocess
24    from dataclasses import dataclass
25    from typing import Dict, List, Optional, Tuple
26
27    from backend.eval.annotations import parse_timestamp
28
29    # Match VideoCompiler defaults so sliced clips frame a rally the same way.
30    BEGIN_PADDING = 0.5
31    END_PADDING = 1.0
32
33
34    @dataclass
35    class ClipWindow:
36        rally: str          # kept as a string to allow inserted ids like "7.5"
37        start: float        # padded, clamped
38        end: float          # padded, clamped
39
40
41    def _read_time(row: Dict[str, str], decimal_key: str, mmss_key: str) -> Optional[float]:
42        """Prefer the decimal-seconds column; fall back to mm:ss. None if unusable."""
43        for key in (decimal_key, mmss_key):
44            val = (row.get(key) or "").strip()
45            if val:
46                try:
47                    return parse_timestamp(val)
48                except ValueError:
49                    continue
50        return None
51
52
53    def load_rally_labels(csv_path: str) -> Dict[str, Tuple[float, float]]:
54        """Map rally_number -> (start, end) seconds, skipping rows with no usable times.
55
56        Accepts the golden schema (start_time/end_time decimal, optional start_mmss/
57        end_mmss). Rally numbers are kept as strings so inserted ids ("7.5") survive.
58        """
59        labels: Dict[str, Tuple[float, float]] = {}
60        with open(csv_path, newline="", encoding="utf-8") as f:
61            reader = csv.DictReader(f)
62            reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
63            [])]
64
65            for row in reader:
66                row = {(k or "").strip().lower(): v for k, v in row.items()}
67                rally = (row.get("rally_number") or "").strip()
68                if not rally:
69                    continue
70                start = _read_time(row, "start_time", "start_mmss")
71                end = _read_time(row, "end_time", "end_mmss")
72                if start is None or end is None or end <= start:

```

```

71         continue # missing/degenerate (e.g. a not-yet-filled MISSED row)
72         labels[rally] = (start, end)
73     return labels
74
75
76 def compute_clip_windows(labels: Dict[str, Tuple[float, float]],
77                           rally_numbers: List[str],
78                           begin_pad: float = BEGIN_PADDING,
79                           end_pad: float = END_PADDING,
80                           video_duration: Optional[float] = None) -> List[ClipWindow]:
81     """Apply padding and clamp to the video, for the requested rallies.
82
83     rally_numbers may be ["all"] to take every labeled rally (in time order).
84     Unknown rally ids are skipped (caller can diff to warn).
85     """
86     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
87         wanted = sorted(labels, key=lambda r: labels[r][0])
88     else:
89         wanted = [r for r in rally_numbers if r in labels]
90
91     windows: List[ClipWindow] = []
92     for rally in wanted:
93         start, end = labels[rally]
94         ps = max(0.0, start - begin_pad)
95         pe = end + end_pad
96         if video_duration is not None and video_duration > 0:
97             if ps >= video_duration:
98                 continue # entirely past the end
99             pe = min(pe, video_duration)
100         if pe <= ps:
101             continue
102         windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103     return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(
109             ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111             capture_output=True, text=True, check=True,
112             ).stdout.strip()
113         return float(out)
114     except (subprocess.SubprocessError, OSError, ValueError):
115         return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []

```



```

136     for w in windows:
137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,
145         ]
146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151             continue
152         if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153             written.append(out_path)
154             print(f"[slicer] rally {w.rally} -> {out_path}")
155     [{w.start:.3f}-{w.end:.3f}s]
156     return written
157
158     def main() -> None:
159         p = argparse.ArgumentParser(description="Slice rally clips from an annotated
160 video.")
161         p.add_argument("--video", required=True, help="Source video file")
162         p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
163         p.add_argument("--rallies", required=True,
164             help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
165         p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
166         p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
167         p.add_argument("--end-pad", type=float, default=END_PADDING)
168         args = p.parse_args()
169
170         rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
171         written = slice_rallies(args.video, args.labels, rally_numbers,
172             args.out_dir, args.begin_pad, args.end_pad)
173         print(f"\nWrote {len(written)} clip(s) to {args.out_dir}")
174
175     if __name__ == "__main__":
176         main()
177

```

13. assistant (2026-06-18T07:48:07.228Z)

Now the remaining two scripts and the training directory.

14. user (2026-06-18T07:48:08.187Z)

```

1     """Phase 3 driver: batch process + evaluate a collector export against ground truth.
2
3     Walks the `manifest.json` produced by sports-data-collector's `curate export`,
4     processes each video through a segmenter (so detections land in the indexer DB),
5     scores each against its ground-truth rally CSV, and prints per-video plus
6     corpus-aggregate precision/recall/F1, mean IoU, and boundary error.
7
8     Examples
9     -----
10    Free CPU baseline on a single video (validate the loop):
11        python run_phase3_eval.py --manifest ../sports-data-
12 collector/data/eval_export/manifest.json \
13        --segmenter motion --only shuttlest_1

```

```

14 Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
15     python run_phase3_eval.py --manifest ../manifest.json --segmenter yolo_hybrid \
16         --json output/phase3_report.json
17
18 Re-score only (videos already processed), no re-segmentation:
19     python run_phase3_eval.py --manifest ../manifest.json --score-only
20 ""
21
22 import os
23 import sys
24 import json
25 import time
26 import argparse
27 import logging
28
29 from dotenv import load_dotenv
30 load_dotenv() # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
31
32 from backend.infrastructure.database import Database
33 from backend.config import load_config
34 from backend.utils.hashing import compute_video_id as get_file_md5 # canonical video_id
35 # Importing the package registers all segmenters (motion/gemini/yolo_hybrid/...).
36 from backend.pipeline.segmenters import segmenter_registry
37 from backend.results import unwrap_or_failure
38 from backend.eval.annotations import load_annotations
39 from backend.eval.harness import evaluate, format_report_text, report_to_dict
40 from backend.eval import batch
41
42 logger = logging.getLogger(__name__)
43
44
45 def build_parser() -> argparse.ArgumentParser:
46     p = argparse.ArgumentParser(
47         description="Batch process + evaluate a collector export manifest (Phase 3).",
48         formatter_class=argparse.RawDescriptionHelpFormatter,
49     )
50     p.add_argument("--manifest", required=True, help="Path to the collector's
manifest.json.")
51     p.add_argument("--videos-root", default=None,
52         help="Root for resolving manifest relative local_paths "
53             "(default: the collector repo root, two levels above the
manifest).")
54     p.add_argument("--segmenter", default="motion",
55         help="Segmenter to run (default: motion ? free/CPU). "
56             "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.")
57     p.add_argument("--stage", choices=("candidates", "final", "both", "auto"),
58         default="auto",
59         help="Which prediction stage(s) to score (default: auto by
segmenter).")
60     p.add_argument("--detector", default=None, help="Restrict candidate scoring to this
detector.")
61     p.add_argument("--iou", type=float, default=0.5, help="IoU match threshold (default:
0.5).")
62     p.add_argument("--only", action="append", default=None,
63         help="Only this video_id (repeatable).")
64     p.add_argument("--limit", type=int, default=None, help="Process at most N videos.")
65     p.add_argument("--max-frames", type=int, default=None,
66         help="Cap segmenter frames (fast validation).")
67     p.add_argument("--score-only", action="store_true",
68         help="Skip segmentation; only score detections already in the DB.")
69     p.add_argument("--reprocess", action="store_true",
70         help="Re-run segmentation even if the video already has segments.")
71     p.add_argument("--db", default=None, help="Indexer DB path (default:
output/<db_name>).")
72     p.add_argument("--json", dest="json_out", default=None, help="Write a JSON report

```

```

here.")
72     p.add_argument("--show-failures", action="store_true", help="List missed/spurious
rallies per video.")
73     return p
74
75
76     def main(argv=None) -> int:
77         logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
78         args = build_parser().parse_args(argv)
79
80         manifest_path = os.path.abspath(args.manifest)
81         if not os.path.exists(manifest_path):
82             print(f"ERROR: manifest not found: {manifest_path}", file=sys.stderr)
83             return 2
84         manifest_dir = os.path.dirname(manifest_path)
85         videos_root = args.videos_root or os.path.dirname(os.path.dirname(manifest_dir))
86
87         with open(manifest_path) as f:
88             manifest = json.load(f)
89             entries = manifest.get("eval_sets", [])
90
91         cfg = load_config() # typed AppConfig (same object the live pipeline feeds
segmenters)
92         db_path = args.db or os.path.join(cfg.output.output_dir, cfg.output.db_name)
93         os.makedirs(os.path.dirname(db_path) or ".", exist_ok=True)
94         db = Database(db_path)
95
96         stages = batch.default_stages_for(args.segmenter, args.stage)
97
98         # Resolve the segmenter class up front (unless purely scoring).
99         segmenter = None
100        if not args.score_only:
101            seg_cls = segmenter_registry.get(args.segmenter)
102            if not seg_cls:
103                print(f"ERROR: segmenter '{args.segmenter}' not found. "
104                    f"Available: {list(segmenter_registry.list_available().keys())}",
file=sys.stderr)
105                return 2
106            segmenter = seg_cls(db, cfg)
107
108        # Filter entries.
109        targets = []
110        for e in entries:
111            if args.only and e["video_id"] not in args.only:
112                continue
113            targets.append(e)
114        if args.limit:
115            targets = targets[: args.limit]
116
117        per_video = []
118        skipped = []
119        print(f"Phase 3: {len(targets)} target(s) | segmenter={args.segmenter} |
stages={stages} | db={db_path}\n")
120
121        for e in targets:
122            vid = e["video_id"]
123            video_path = batch.resolve_video_path(e.get("local_path"), videos_root)
124            csv_path = os.path.join(manifest_dir, e["csv"])
125
126            if not video_path or not os.path.exists(video_path):
127                skipped.append((vid, "no video file"))
128                print(f"[skip] {vid}: no video file ({e.get('local_path')})")
129                continue
130            if not os.path.exists(csv_path):
131                skipped.append((vid, "no GT csv"))

```

```

132         print(f"[skip] {vid}: GT csv missing ({csv_path})")
133         continue
134
135     video_id = get_file_md5(video_path)
136
137     # Segment unless score-only or already present (and not --reprocess).
138     if not args.score_only:
139         already = db.query_play_segments(video_id, {}) or
db.query_candidate_segments(video_id)
140         if already and not args.reprocess:
141             print(f"[have] {vid}: detections already in DB (use --reprocess to
redo)")
142         else:
143             print(f"[proc] {vid}: segmenting with {args.segmenter}...")
144             db.add_video(video_id, video_path, "processed", [])
145             t0 = time.time()
146             # process_video returns a Result (Success|Failure) for some segmenters
147             # (e.g. the default `motion`); unwrap before len() so this doesn't crash
with
148             # TypeError on a Success dataclass ? mirrors the unwrap in
backend/main.py.
149             segs, failure = unwrap_or_failure(
150                 segmenter.process_video(video_id, video_path,
max_frames=args.max_frames))
151             if failure is not None:
152                 print(f"        -> FAILED: {failure.message}")
153                 skipped.append((vid, f"segment failed: {failure.message}"))
154                 continue
155             print(f"        -> {len(segs)} segments in {time.time() - t0:.0f}s")
156
157         gt = load_annotations(csv_path)
158         report = evaluate(db, video_id, gt, stages, iou_threshold=args.iou,
detector=args.detector)
159         print(format_report_text(report, show_failures=args.show_failures))
160         print()
161         d = report_to_dict(report)
162         d["collector_video_id"] = vid
163         per_video.append(d)
164
165     agg = batch.aggregate(per_video, stages)
166     print(batch.format_aggregate(agg))
167     if skipped:
168         print(f"\nSkipped {len(skipped)}: " + ", ".join(f"{v} ({why})" for v, why in
skipped))
169
170     if args.json_out:
171         with open(args.json_out, "w") as f:
172             json.dump({"segmenter": args.segmenter, "iou": args.iou, "stages": stages,
173                 "aggregate": agg, "per_video": per_video,
174                 "skipped": skipped}, f, indent=2)
175         print(f"\nJSON report written to {args.json_out}")
176
177     return 0
178
179
180 if __name__ == "__main__":
181     raise SystemExit(main())
182

```

15. user (2026-06-18T07:48:08.645Z)

```

1     import os
2     import json
3     import logging
4     from dotenv import load_dotenv

```

```

5
6     load_dotenv()
7
8     # Central root-logger setup ? all backend modules use logging.getLogger(__name__).
9     # Quiet third-party HTTP chatter by default; re-honoured against the config knob
10    # (logging.verbose_http) once the config is loaded in main().
11    from backend.utils.logging_setup import setup_logging
12    setup_logging()
13
14    from backend.infrastructure.database import Database
15    from backend.config import load_config
16    from backend.utils.hashing import compute_video_id
17    from backend.pipeline.segmenters import segmenter_registry
18    from backend.pipeline.stitcher.compiler import VideoCompiler
19    from backend.results import unwrap_or_failure
20    from backend.tools.export import format_rally_summary
21
22    logger = logging.getLogger(__name__)
23
24    def main():
25        video_path = r"C:\Users\avidu\Videos\Testing\KushagraYashAviFirstVideo.mp4"
26        output_dir = r"C:\Users\avidu\Videos\Testing"
27        output_filename = "Final_2_OutputOfKushagraYashAviFirstVideo.mp4"
28
29        print("Loading config...")
30        cfg = load_config() # typed AppConfig ? single source of defaults
31        # Re-apply now that the verbose_http knob is known (default keeps HTTP chatter
32        quiet).
33        setup_logging(verbose_http=cfg.logging.verbose_http)
34
35        print("Verifying files exist...")
36        if not os.path.exists(video_path):
37            print(f"Error: Video file not found at {video_path}")
38            return
39
40        os.makedirs(cfg.output.output_dir, exist_ok=True)
41        db_path = os.path.join(cfg.output.output_dir, cfg.output.db_name)
42        db = Database(db_path)
43
44        # Calculate MD5 of the video to uniquely identify it
45        import time
46        print("Calculating video MD5 (this might take a few seconds on large files)...",
47        flush=True)
48        t0 = time.time()
49        video_id = compute_video_id(video_path)
50        print(f"Calculated MD5: {video_id} (took {time.time()-t0:.1f}s)", flush=True)
51
52        segments = []
53        cached_video = db.get_video(video_id)
54        cache_hit = False
55
56        if cached_video and cached_video.get("status") == "processed":
57            segments = db.query_play_segments(video_id, {})
58            if len(segments) > 0:
59                print(f"\n[CACHE] Found cached indexed video in database (MD5: {video_id})
60                with {len(segments)} parsed play segments.", flush=True)
61                choice = input("Would you like to (1) Run stitching on the cached segments,
62                or (2) Reprocess the video with Gemini from scratch? [1/2]: ").strip()
63                if choice == '1':
64                    print("[CACHE] Using cached play segments. Skipping segmenter API
65                    call.", flush=True)
66                    cache_hit = True
67                else:
68                    print("[CACHE] Ignoring cache...", flush=True)
69            else:

```

```

65         print("[CACHE] Video exists in DB but contains 0 play segments. Treating as
cache miss.", flush=True)
66
67     if not cache_hit:
68         print("[CACHE] Initializing clean indexing...", flush=True)
69         db.clear_video_data(video_id)
70
71         print("Bypassing OpenCV validations for performance on massive files...",
flush=True)
72         db.add_video(video_id, video_path, "processed", [])
73
74         print("Running segmenter & indexing...", flush=True)
75         seg_name = cfg.indexing.default_segmenter
76         segmenter_cls = segmenter_registry.get(seg_name)
77         if not segmenter_cls:
78             print(f"[FAIL] Segmenter '{seg_name}' not found.")
79             return
80         print(f"Using segmenter: {seg_name}", flush=True)
81         segmenter = segmenter_cls(db, cfg)
82         # process_video returns a Result (Success|Failure) for some segmenters (e.g.
`motion`);
83         # unwrap before len()/truthiness so this doesn't crash with TypeError on a
Success
84         # dataclass (and so the empty-guard below actually sees the list) ? mirrors
backend/main.py.
85         segments, failure = unwrap_or_failure(segmenter.process_video(video_id,
video_path))
86         if failure is not None:
87             print(f"[FAIL] Segmentation failed: {failure.message}")
88             return
89         print(f"Successfully indexed {len(segments)} play segments.", flush=True)
90
91     if not segments:
92         print("No play segments detected. Cannot compile highlights.")
93         return
94
95
96     end_padding = cfg.stitching.end_padding
97     begin_padding = cfg.stitching.begin_padding
98     min_shot_count = cfg.stitching.min_shot_count
99
100    if cache_hit:
101        print("\n--- Stitching Customizations ---")
102        try:
103            bpad_input = input(f"Enter begin_padding in seconds (default
{begin_padding}): ").strip()
104            if bpad_input:
105                begin_padding = float(bpad_input)
106
107            pad_input = input(f"Enter end_padding in seconds (default {end_padding}):
").strip()
108            if pad_input:
109                end_padding = float(pad_input)
110
111            shot_input = input(f"Enter min_shot_count to filter by (default
{min_shot_count}): ").strip()
112            if shot_input:
113                min_shot_count = int(shot_input)
114        except ValueError:
115            print("[WARNING] Invalid input detected, falling back to defaults from
config.json.")
116
117    print(f"\nCompiling highlights (with begin_padding={begin_padding}s,
end_padding={end_padding}s, min_shot_count={min_shot_count})...", flush=True)
118

```

```

119     # Filter segments where shot_count >= min_shot_count
120     time_pairs = []
121     skipped_count = 0
122     for s in segments:
123         meta = s.get("metadata", {})
124         if isinstance(meta, str):
125             try:
126                 meta = json.loads(meta)
127             except Exception:
128                 meta = {}
129         shot_count = meta.get("shot_count", 0)
130         if shot_count >= min_shot_count:
131             time_pairs.append((s["start_time"], s["end_time"]))
132         else:
133             skipped_count += 1
134
135     if skipped_count > 0:
136         print(f"[FILTER] Excluded {skipped_count} play segments with fewer than
137 {min_shot_count} shots.", flush=True)
138     print(f"[FILTER] Retaining {len(time_pairs)} play segments for stitching.",
139 flush=True)
140
141     # End-of-run rally table for manual QA ? sourced from the produced play_segments so
142     # it lines up row-for-row with the rally CSV. Logged as one block (single timestamp,
143     # aligned rows) so it survives even if the table is the last thing the owner scans.
144     def _log_rally_summary() -> None:
145         logger.info("\n%s", format_rally_summary(segments))
146
147     if not time_pairs:
148         print("No play segments passed the filtering criteria. Cannot compile
149 highlights.")
150         _log_rally_summary()
151         return
152
153     compiler = VideoCompiler(output_dir, end_padding=end_padding,
154 begin_padding=begin_padding,
155                             watermark=cfg.stitching.watermark)
156     try:
157         out_path = compiler.compile_highlights(video_path, time_pairs, output_filename)
158         print(f"Stitched highlight video saved to: {out_path}", flush=True)
159     except Exception as e:
160         print(f"Error during stitching: {e}", flush=True)
161
162     _log_rally_summary()
163
164 if __name__ == "__main__":
165     main()
166

```

16. assistant (2026-06-18T07:48:10.695Z)

Now let me look at the training directory and the spec.yaml.

17. user (2026-06-18T07:48:12.165Z)

```

training/gen0/harness.py
training/gen0/__init__.py
training/README.md
training/requirements-train.txt
training/__init__.py

```

18. user (2026-06-18T07:48:14.573Z)

```

1     """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->

```

```

artifact.
2
3     python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4         --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
5         --version 0.1.0
6     Productionizes the rally_seq prototype per ADR-008:
7     - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8       honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9       the FINAL model on all videos.
10    - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11        <out>/<version>/weights.json    (gitignored; classifier + frozen normalization)
12        <out>/<version>/manifest.json    (the reviewable contract: feature-schema version,
13                                          calibration, training lineage + GT hashes,
14                                          eval results, attestations, gate verdict)
15    - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16      (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17      (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18      undefined => no self-declared victory). The product side owns final admission
19      (backend/eval/regression.py); this harness never grades itself into serving.
20
21    Training may import backend.* (one-way wall); backend must never import training.*.
22    """
23    from __future__ import annotations
24
25    import argparse
26    import hashlib
27    import json
28    import os
29    import subprocess
30    import time
31    from math import comb
32    from typing import Dict, List, Optional
33
34    import numpy as np
35
36    from backend.eval.classifier import LogisticRegression
37    from backend.eval.gt_loader import load_gt_intervals
38    from backend.eval.regression import gt_hash
39    from backend.eval.rally_seq_proto import (
40        labels_for, run as lovo_run, _seg_f1,
41    )
42    from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
43    features
44    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
45
46    # Post-processing constants ? part of the model's identity, recorded in the manifest.
47    CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
48    THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]
49
50    def _git_sha() -> str:
51        try:
52            return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
53                                  text=True, timeout=10).stdout.strip() or "unknown"
54        except Exception:
55            return "unknown"
56
57
58    def _sha256(path: str) -> str:
59        h = hashlib.sha256()
60        with open(path, "rb") as f:
61            for chunk in iter(lambda: f.read(65536), b''):
62                h.update(chunk)
63        return h.hexdigest()

```



```

64
65
66 def sign_test(learned: List[float], floor: List[float]):
67     """Paired one-sided sign test: is learned > floor more often than chance?
68
69     At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
70     wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
71     pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv] # ties drop,
standard
72     wins = sum(1 for lv, fv in pairs if lv > fv)
73     n = len(pairs)
74     p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
75     return wins, n, p
76
77
78 def train_final(specs: List[Dict]):
79     """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).
80
81     The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
82     vids = []
83     for s in specs:
84         pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
85         arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
86         X, times = features(arr, float(s["fps"]))
87         gts = load_gt_intervals(s["labels"], strict=False)
88         vids.append({"name": s["name"], "X": X, "times": times,
89                     "y": labels_for(times, gts), "gts": gts})
90     Xall = np.vstack([v["X"] for v in vids])
91     yall = np.concatenate([v["y"] for v in vids])
92     clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
93     clf.fit(Xall, yall, feature_names=FEAT_NAMES)
94     thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
95     return clf, float(thr)
96
97
98 def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
99     """Pre-flight ship-gate.
100
101     Provisional F1 target (NOT owner-ratified; calibration tracked in #172).
102     Floor (necessary): heuristic LOVO ~0.480 (from quality table).
103     Multi-court ref (makes owned \"worth serving\" per data): 0.66.
104     Proposed for \"very high P/R\" + significance: F1@IoU0.5 >= 0.70 on
105     multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
106     Beating floor + sign test necessary; C3 main commercial blocker.
107     ship=True only when all non-C3 blocks clear (honesty first).
108     """
109     reasons: List[str] = []
110     floor_passed = None
111     sign = None
112     if floor:
113         floor_mean = float(floor["mean_f1"])
114         floor_passed = eval_res["mean_f1"] > floor_mean
115         if not floor_passed:
116             reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
117         per = floor.get("per_video", {})
118         paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
119         if paired:
120             wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])
121             sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
122             if p >= 0.05:
123                 reasons.append(f"paired sign test not significant ({wins}/{n} wins,

```

```

p={p:.3f}) "
124                                     f"? grow the corpus before claiming superiority")
125     else:
126         reasons.append("no floor baseline provided ? nothing to beat")
127
128     # Provisional blocker (per owner review on #170; see #172 for calibration).
129     # (The number itself is not the blocker; calibration tracked in #172.)
130     reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
#172")
131     reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
regardless of F1")
132     # ship remains False until C3 resolved + other gates (by design).
133     return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
reasons}
134
135
136     def run(manifest_path: str, out_dir: str, version: str,
137            floor_path: Optional[str] = None) -> dict:
138         with open(manifest_path, encoding="utf-8") as f:
139             specs = json.load(f)
140             floor = None
141             if floor_path:
142                 with open(floor_path, encoding="utf-8") as f:
143                     floor = json.load(f)
144
145             # 1) Honest LOVO eval (the number that matters).
146             eval_res = lovo_run(specs)
147
148             # 2) Final model on all videos.
149             clf, thr = train_final(specs)
150
151             # 3) Gate (pre-flight).
152             verdict = gate_verdict(eval_res, floor)
153
154             # 4) Artifact bundle.
155             bundle_dir = os.path.join(out_dir, version)
156             os.makedirs(bundle_dir, exist_ok=True)
157             weights_path = os.path.join(bundle_dir, "weights.json")
158             payload = clf.to_dict()
159             payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
160             payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
161             with open(weights_path, "w", encoding="utf-8") as f:
162                 json.dump(payload, f, indent=2)
163
164             manifest = {
165                 "artifact": {
166                     "name": "rally_tal_gen0", "version": version,
167                     "created_at": time.strftime("%Y-%m-%dT%H:%M:%SZ"),
168                     "git_sha": _git_sha(),
169                     "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
170                 },
171                 "feature_schema": FEATURE_SCHEMA,
172                 "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
173                                "note": "threshold tuned train-side on the full corpus; "
174                                "honest generalization = the LOVO eval below"},
175                 "training_lineage": {
176                     "corpus_manifest": manifest_path,
177                     "split_protocol": "leave-one-video-out by match",
178                     "label_source": "human (rally-annotator golden CSVs)",
179                     "videos": [{"name": s["name"], "labels": s["labels"],
180                               "gt_hash": gt_hash(load_gt_intervals(s["labels"],
strict=False))},
181                                {"fps": s["fps"]} for s in specs],
182                 },
183                 "eval": {

```

```

184         "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
185         "per_video": eval_res["per_video"],
186         "mean_f1": eval_res["mean_f1"],
187         "floor": floor,
188     },
189     "attestation": {
190         "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
191         "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
192         "no_paid_llm_training_labels": True,
193         "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
194     },
195     "gate": verdict,
196 }
197 manifest_out = os.path.join(bundle_dir, "manifest.json")
198 with open(manifest_out, "w", encoding="utf-8") as f:
199     json.dump(manifest, f, indent=2)
200
201 print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
202 print(f"weights : {weights_path}")
203 print(f"manifest: {manifest_out}")
204 print(f"GATE      ship={verdict['ship']} floor_passed={verdict['floor_passed']} "
205       f"sign_test={verdict['sign_test']}")
206 for b in verdict["blockers"]:
207     print(f" - {b}")
208 return manifest
209
210
211 def main() -> None:
212     ap = argparse.ArgumentParser(description=__doc__)
213     ap.add_argument("--manifest", required=True,
214                    help="corpus manifest JSON:
[{'name,trajectory,fps,frame_width,labels}, ...]")
215     ap.add_argument("--floor", default=None,
216                    help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
217     ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
218     ap.add_argument("--version", required=True, help="semver for the artifact bundle")
219     args = ap.parse_args()
220     run(args.manifest, args.out, args.version, floor_path=args.floor)
221
222
223 if __name__ == "__main__":
224     main()
225

```

19. assistant (2026-06-18T07:48:17.965Z)

Now I have a good picture. Let me dig into specific suspect areas. Let me check the harvest divergence logic, the `format\_aggregate` for empty `per\_video`, and verify a few contracts against tests. First, let me look at the test for nightly_regression to understand the contract, and at `batch.aggregate`.

20. user (2026-06-18T07:48:20.185Z)

```

tests/test_eval_harness.py
tests/test_export_wasb_segments.py
tests/test_nightly_regression.py
tests/test_rally_slicer.py
tests/test_reporting_harvest.py
tests/test_training_gen0.py

```

21. user (2026-06-18T07:48:23.392Z)

test_run_signals.py

22. assistant (2026-06-18T07:48:27.319Z)

Let me examine the `compute\_clip\_windows` "all" sort and a potential issue in `\_read\_time` for `start\_mmss`/`end\_mmss` column lookup. Also check the export.py CSV: `segments\_to\_csv` doesn't parse JSON-string metadata (unlike `\_segment\_metadata`). Let me look at the rally_slicer test and the harness test to understand expected contracts.

23. user (2026-06-18T07:48:27.806Z)

```
1      """Tests for the rally slicer's pure logic (no ffmpeg)."""
2      import pytest
3
4      from backend.tools.rally_slicer import (
5          load_rally_labels, compute_clip_windows, ClipWindow, BEGIN_PADDING, END_PADDING,
6      )
7      from backend.tools.export import basic_stats, segments_to_json, segments_to_csv,
8      segments_to_edl
9
10     def _write(tmp_path, rows,
11 header="rally_number,start_time,end_time,start_mmss,end_mmss"):
12         p = tmp_path / "labels.csv"
13         p.write_text(header + "\n" + "\n".join(rows) + "\n", encoding="utf-8")
14         return str(p)
15
16     # ----- load_rally_labels
17
18     def test_load_decimal_and_mmss(tmp_path):
19         csv = _write(tmp_path, [
20             "1,3.0,21.0,,", # decimal
21             "2,,0:29.000,0:32.000", # mm:ss fallback
22         ])
23         labels = load_rally_labels(csv)
24         assert labels["1"] == pytest.approx((3.0, 21.0))
25         assert labels["2"] == pytest.approx((29.0, 32.0))
26
27
28     def test_load_skips_missing_and_degenerate(tmp_path):
29         csv = _write(tmp_path, [
30             "7.5,106.031,,", # MISSED row: no end -> skipped
31             "17,237.0,237.0,,", # zero-duration -> skipped
32             "3,44.5,51.0,,", # good
33         ])
34         labels = load_rally_labels(csv)
35         assert "7.5" not in labels and "17" not in labels
36         assert labels["3"] == pytest.approx((44.5, 51.0))
37
38
39     def test_fractional_ids_preserved(tmp_path):
40         csv = _write(tmp_path, ["16.5,214.0,223.0,,"])
41         assert "16.5" in load_rally_labels(csv)
42
43
44     # ----- compute_clip_windows
45
46     def test_padding_applied():
47         w = compute_clip_windows({"1": (10.0, 20.0)}, ["1"])
48         assert w == [ClipWindow("1", 10.0 - BEGIN_PADDING, 20.0 + END_PADDING)]
49
50
51     def test_clamp_to_video_bounds():
52         # start near 0 clamps to 0; end past duration clamps to duration
```

```

53     w = compute_clip_windows({"1": (0.2, 99.0)}, ["1"], video_duration=99.5)
54     assert w[0].start == 0.0
55     assert w[0].end == 99.5
56
57
58     def test_all_selects_every_rally_in_time_order():
59         labels = {"2": (29.0, 32.0), "1": (3.0, 21.0)}
60         w = compute_clip_windows(labels, ["all"])
61         assert [x.rally for x in w] == ["1", "2"]    # ordered by start time, not dict order
62
63
64     def test_subset_and_unknown_skipped():
65         labels = {"1": (3.0, 21.0), "2": (29.0, 32.0), "3": (44.5, 51.0)}
66         w = compute_clip_windows(labels, ["3", "1", "99"]) # 99 unknown -> dropped
67         assert [x.rally for x in w] == ["3", "1"]        # preserves requested order
68
69
70     def test_window_entirely_past_end_skipped():
71         w = compute_clip_windows({"1": (200.0, 210.0)}, ["1"], video_duration=100.0)
72         assert w == []
73
74
75     # ----- Value Layer exports
76     (minimal slice)
77
78     def _segs():
79         return [
80             {"id": 1, "start_time": 10.0, "end_time": 15.0, "duration": 5.0, "metadata":
81             {"shot_count": 4, "detector": "wasb-hybrid-gemini"}},
82             {"id": 2, "start_time": 30.0, "end_time": 33.5, "duration": 3.5, "metadata":
83             {"shot_count": 3, "detector": "wasb-hybrid-gemini"}},
84         ]
85
86     def test_basic_stats_empty_and_nonempty():
87         assert basic_stats([])["count"] == 0
88         s = basic_stats(_segs())
89         assert s["count"] == 2
90         assert s["total_duration_s"] == 8.5
91         assert s["avg_duration_s"] == 4.25
92
93     def test_segments_to_json_roundtrips_and_includes_stats():
94         j = segments_to_json(_segs(), video_id="v1", video_path="clip.mp4")
95         assert j["video_id"] == "v1"
96         assert len(j["segments"]) == 2
97         assert j["stats"]["count"] == 2
98
99     def test_segments_to_csv_has_header_and_data():
100         csv = segments_to_csv(_segs())
101         assert csv.startswith("start_time,end_time")
102         assert "10.0,15.0,5.0,4" in csv
103
104
105     def test_segments_to_edl_produces_plausible_title_and_clip_lines():
106         edl = segments_to_edl(_segs(), title="MyReel")
107         assert "TITLE: MyReel" in edl
108         assert "AX V C" in edl
109         assert "FROM CLIP NAME" in edl
110

```

24. assistant (2026-06-18T07:48:39.508Z)

Now let me focus on the `compute\_clip\_windows` "all" case. There is a potential issue: when

`rally\_numbers == ["all"]` is checked with `.lower()`, but what if `rally\_numbers` is `["ALL", "1"]`? Then it's not length 1 so falls into the else, and "ALL" won't match any label so dropped ? that's fine.

Let me examine the harvest `matches\_default` logic and the `\_segment\_metadata` discrepancy in `segments\_to\_csv`. Note: `segments\_to\_csv` reads `s.get("metadata")` directly without JSON-parsing, while `\_segment\_metadata` handles the string case. This is a real inconsistency for DB round-trip data, but let me verify whether DB segments would have string metadata. Let me check the `run\_phase3\_eval.py` more carefully ? there's a subtle bug around the `--only` filter and `e["video\_id"]`.

Let me look more carefully at a potential off-by-one or KeyError. In `run\_phase3\_eval.py` line 111: `e["video\_id"]` ? if a manifest entry lacks `video\_id`, it'd KeyError. But that's the manifest contract. Let me focus on the divergence logic in harvest and check `nightly\_regression` exit codes against the docstring.

The docstring says exit code `4` = baseline STALE. Let me verify: `exit\_code()` returns 4 for stale. But the docstring also describes `3` = no baseline. In `main`, when `baseline is None`, returns 3. Good. Let me check a subtle issue: the `--update-baseline` path computes `snapshot` but doesn't check `args.update\_baseline` before requiring the manifest. That's fine.

Let me look at a potential real bug: in `nightly\_regression.main`, when `args.update\_baseline` is set but manifest is missing, it returns 2 before. Fine. Let me re-examine `compare\_all` ? it iterates `cur\_cfgs` but a config in baseline but NOT in current is silently ignored (a removed config). That's arguably intended.

Let me now check the `\_ai\_stage` `provider.upper()` ? if `config.get("ai\_provider")` is an empty string, `("")` or "gemini" ? "gemini". Fine.

Let me verify the EDL timecode rounding bug and the `segments\_to\_csv` JSON-string issue against actual DB output shape.

25. user (2026-06-18T07:48:41.058Z)

```
409:     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) -> List[Dict[str,
Any]]:
410:         """
411:         Dynamically queries play segments based on filters:
412:         - duration_min: float
413:         - server: string
414:         - receiver: string
415:         - event_type: string (e.g. smash, foul)
416:         """
417:         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
418:         params = [video_id]
419:
420:         if filters.get("duration_min"):
421:             query += " AND r.duration >= ?"
422:             params.append(filters["duration_min"])
423:
424:         if filters.get("server"):
425:             query += " AND r.server = ?"
426:             params.append(filters["server"])
427:
428:         if filters.get("receiver"):
429:             query += " AND r.receiver = ?"
430:             params.append(filters["receiver"])
431:
432:         if filters.get("event_type"):
433:             # Check if there is an event of a specific type inside this segment
434:             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
435:             params.append(filters["event_type"])
436:
437:         segments_list = []
```

```

438-         with self._connect() as conn:
439-             cursor = conn.cursor()
440-             rows = cursor.execute(query, params).fetchall()
441-
442-             for row in rows:
443-                 r_dict = dict(row)
444-                 r_id = r_dict["id"]
445-                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
446-
447-                 # Fetch associated events
448-                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
449-                 r_dict["events"] = []

```

26. assistant (2026-06-18T07:48:48.985Z)

So `query\_play\_segments` already JSON-decodes `metadata` to a dict. So the DB round-trip yields dict metadata, meaning `segments\_to\_csv` reading `s.get("metadata")` directly works for DB segments. The `\_segment\_metadata` string-handling in export.py is for robustness. Not a bug.

Let me now look more carefully at the harvest divergence logic ? there's a documented concern. The comment says "diverges only when a segmenter actually RAN, the user did NOT explicitly pick it". But the code sets `matches\_default = (segmenter\_actual == config\_default)` regardless of whether the user explicitly requested it. Let me check what the spec.yaml rule keys on.

27. user (2026-06-18T07:48:49.147Z)

```

1      # Observability report spec for the badminton-highlight-indexer.
2      # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3      # library; this file is the indexer-specific layout + footgun rules. Every
4      # actionable rule cites a README/FAQ section so a reader can self-debug.
5      name: indexer
6      title: Badminton Highlight Indexer ? Video Report
7
8      # Customer here = whoever receives the highlight reel.
9      customer_verdict:
10         pass: "? Your highlights are ready."
11         pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12         fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14         sections:
15             - { id: run,          title: "Run",          kind: identity,  audience: both }
16             - { id: video,       title: "Video",        kind: video_meta, audience: both }
17             - { id: highlights,   title: "Highlights",    kind: highlights, audience: customer
}
18             - { id: cust_notes,  title: "Things to know", kind: flags,      audience: customer
}
19             - { id: stages,      title: "Processing stages", kind: stages,    audience:
technical }
20             - { id: findings,    title: "Findings & warnings", kind: flags,      audience:
technical }
21             - { id: verdict,     title: "Verdict",        kind: verdict,   audience: both }
22
23         rules:
24             # The classic silent failure: 0 segments because the AI provider no-op'd on a
25             # missing API key ? NOT because the video has no rallies.
26             - code: silent_provider_noop
27               severity: error
28               when:
29                 - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
30                 - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
31             message: >-
32                 0 segments AND no AI provider API key was set. The segmenter (see Processing
33                 stages > segmentation > segmenter_actual) hands its candidate windows to the AI

```

```

34     provider, which silently did nothing without a key ? so this is a MISSING
35     CREDENTIAL, not 'the video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a
36     .env file in the project root and re-run (see README#Q7).
37     faq_ref: "README#Q7"
38     customer_message: >-
39         We couldn't generate highlights for this video due to a configuration issue
40         on our side. Please contact support.
41
42     # The provider REFUSES clips over its upload cap (providers/gemini.py::MAX_UPLOAD_MB)
43     # and returns no segments for them. A high-bitrate (4K) source whose chunks were
44     # stream-copied can lose EVERY chunk this way and still end "0 segments / success" ?
45     # observed live 2026-06-10. The count flows segmenter -> run_signals -> ai_handoff.
46     - code: oversize_clip_skipped
47       severity: error
48       when:
49         - { path: "stages.ai_handoff.details.oversize_clips_skipped", op: gt, value: 0 }
50       message: >-
51         One or more extracted chunks exceeded the AI provider's upload cap and were
52         SKIPPED WITHOUT ANALYSIS ? segments are missing for those time ranges, so a
53         low/zero segment count does NOT mean 'no rallies'. This happens when a
54         high-bitrate (e.g. 4K) source is chunked via stream-copy. Fix: keep
55         indexing.ai_chunk_scale_width at its default (1280) so AI chunks are
56         re-encoded/downscaled before upload (0 disables re-encoding and reintroduces
57         this failure), then re-run.
58     faq_ref: "README#Q7"
59     customer_message: >-
60         Parts of this video were too large to analyze, so some highlights may be
61         missing. Please contact support.
62
63     # API key WAS present but an AI segmenter still produced nothing ? likely a
64     # provider error / quota / WSL healthcheck issue rather than a truly empty video.
65     - code: ai_empty_vs_no_rallies
66       severity: warn
67       when:
68         - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
69         - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
70         - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
71       message: >-
72         An AI-based segmenter returned 0 segments even though the API key is set.
73         Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
74         failed WSL healthcheck ? these all return [] silently. Re-run with -u for
75         unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
76     faq_ref: "README#Q7"
77
78     # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
79     - code: fps_fallback_used
80       severity: warn
81       when:
82         - { path: "video_meta.fps_source", op: eq, value: fallback }
83       message: >-
84         fps was NOT probed from the file ? a fallback default (shown as the fps value)
85         was used. Because rally timing and motion thresholds scale with fps, boundaries
86         and motion sensitivity may be miscalibrated. This usually means validation didn't
87         run (it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real
88         fps with ffprobe and re-run.
89     faq_ref: "README#Q14"
90
91     # Audio stream missing or disabled. Per VIDEO_RECORDING_GUIDELINES (and ADR-007)
92     # the shuttle "thwack" is a first-class rally cue for recall; recording with audio
93     # on (mic unobstructed, no aggressive noise suppression) is now a hard requirement
94     # for good results with the shuttle-tracking substrates.
95     - code: audio_disabled_or_missing
96       severity: warn
97       when:
98         - { path: "video_meta.audio_present", op: eq, value: false }

```



```

99     message: >-
100         No audio stream was detected in the input (or it could not be read). The shuttle
101         "thwack" impact sound is a strong cue that improves rally recall (especially for
102         the WASB/TrackNet detector paths). Record with audio enabled, keep the camera
103         mic unobstructed, and avoid heavy wind/noise-reduction filters that flatten the
104         transient. See VIDEO_RECORDING_GUIDELINES.md (the "Audio (REQUIRED)" row) and
105         AUDIO_RALLY_DETECTION_PLAN.md.
106     faq_ref: "VIDEO_RECORDING_GUIDELINES"
107     customer_message: >-
108         For best highlight results, record with audio on and the mic unobstructed.
109         The shuttle impact sound helps the detector find rallies. Re-record or
110         re-process after enabling audio (see VIDEO_RECORDING_GUIDELINES.md).
111
112     # The motion baseline fabricates its per-segment metadata/events ? they are not
113     # measured. Warn so nobody trusts them as ground truth.
114     - code: synthetic_motion_metadata
115       severity: warn
116       when:
117         - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
118     message: >-
119         The 'motion' segmenter was used: its per-segment metadata (intensity, speed,
120         events, server/receiver) are PLACEHOLDER values (unmeasured) ? fabricated by the
121         heuristic, NOT derived from the video ? so a PASS with segments here does not mean
122         the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
123     faq_ref: "README#Q10"
124
125     # The segmenter that ran differs from config's default -> you may be looking at a
126     # different detector than expected (config/setup/CLI defaults are known to diverge).
127     - code: segmenter_divergence
128       severity: info
129       when:
130         - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
false }
131         - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
value: false }
132     message: >-
133         The segmenter that ran differs from config.json's default_segmenter, and no
134         explicit --segmenter was given ? so a fallback default kicked in. The indexer's
135         defaults are known to diverge across config.json / scripts/doctor.py / the CLI;
confirm
136         you ran the detector you meant.
137     faq_ref: "README#Q7"
138
139     # Re-processing an already-indexed video replaced its prior segments (INSERT OR
140     # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
141     - code: reingest_replaced_prior
142       severity: info
143       when:
144         - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
145     message: >-
146         This video was already in the database; re-processing REPLACED its previous
147         segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
148         when you intend to re-run; just know the earlier results are gone.
149     faq_ref: "README#Q11"
150
151     # A validation check failed -> segmentation was halted. Point the reader at the
152     # report's Validation section + the per-check FAQ.
153     - code: validation_failed
154       severity: error
155       when:
156         - { path: "stages.validation.status", op: eq, value: error }
157     message: >-
158         One or more validation checks failed, so indexing was halted. See the
159         Processing stages > validation block above: each failed check states the
160         reason (relevance -> README#Q2, fps/keyframes -> README#Q5).

```

```
161         faq_ref: "README#Q13"
162         customer_message: >-
163             This video couldn't be processed because it didn't pass our quality checks
164             (for example: too blurry, too low resolution, or not a badminton court).
165
```

28. assistant (2026-06-18T07:49:00.724Z)

The divergence rule keys on BOTH ``matches_config_default == false` AND `segmenter_explicitly_requested == false`, so the rule correctly only fires when not explicitly requested. The `matches_default` being computed regardless of explicit request is fine because the spec gates on `segmenter_explicitly_requested` separately. Not a bug.`

Now let me look carefully at a potential real issue in ``nightly_regression.py``. The ``compare_config`` per-video guard at line 307: ``if c > b + tols.over_seg_tol``. With ``over_seg_tol = 0`` default, this is ``c > b``. The ``b`` and ``c`` are converted to float in the Finding but the comparison uses raw values from the dict. For ``split_count`/`merge_count`` those are ints. Fine.

Let me check the EDL ``_tc`` frame rounding for a subtle off-by-one when fractional seconds round to 30 frames. Consider ``t = 0.999``: ``f = int((0.999 - 0) * 30) = int(29.97) = 29``. Max frame is 29 at 30fps. For ``t = 1.9999``: ``int(t)=1``, ``(1.9999-1)*30 = 29.997`` ? 29. So ``f`` is always 0-29. No off-by-one. Fine.

Now let me examine ``run_phase3_eval.py`` more carefully. Line 139: ``db.query_play_segments(video_id, {})` or ``db.query_candidate_segments(video_id)`` ? uses ``video_id`` (MD5) computed at line 135. Good. Line 144 ``db.add_video`` always runs even when reprocessing ? fine.

Let me look at a subtle issue: in ``run_phase3_eval``, ``--only`` filters on ``e["video_id"]`` (collector id ``vid``), but then ``video_id = get_file_md5(...)`` is the MD5. The ``--only`` matches collector IDs ? consistent with the help text "Only this video_id". OK.

Let me now scrutinize the ``training/gen0/harness.py`` ``sign_test`` and ``gate_verdict`` for correctness, and check the ``_seg_f1`` import. Let me check ``rally_seq_proto``.

29. user (2026-06-18T07:49:02.797Z)

```
backend/eval/rally_seq_proto.py:53:def labels_for(times: np.ndarray, gts: List[Interval]) ->
np.ndarray:
backend/eval/rally_seq_proto.py:101:def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
backend/eval/rally_seq_proto.py:109:def run(specs: List[Dict]) -> dict:
backend/eval/gt_loader.py:44:def load_gt_intervals(csv_path: str, *, strict: bool = True) ->
List[Interval]:
```

30. user (2026-06-18T07:49:06.544Z)

```
101     def _seg_f1(clf, vids: List[Dict], thr: float) -> float:
102         f1s = []
103         for v in vids:
104             segs = probs_to_segments(clf.predict_proba(v["X"]), v["times"], thr)
105             f1s.append(score(segs, v["gts"], 0.5).f1)
106         return sum(f1s) / max(len(f1s), 1)
107
108
109     def run(specs: List[Dict]) -> dict:
110         data = {}
111         for s in specs:
112             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
113             arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
114             X, times = features(arr, float(s["fps"]))
115             gts = load_golden_gts(s["labels"])
116             data[s["name"]] = {"X": X, "times": times, "y": labels_for(times, gts), "gts":
gts}
117         names = list(data)
```

```

118     print(f"=== Learned per-frame rally segmenter ? LEAVE-ONE-VIDEO-OUT ({len(names)}
videos) ===")
119     print(f"features: {FEAT_NAMES}\n")
120
121     results = []
122     for test in names:
123         train = [data[n] for n in names if n != test]
124         Xtr = np.vstack([v["X"] for v in train])
125         ytr = np.concatenate([v["y"] for v in train])
126         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
127         clf.fit(Xtr, ytr, feature_names=FEAT_NAMES)
128         # tune the decision threshold on TRAIN segment-F1 (no test leakage)
129         thr = max(np.arange(0.2, 0.81, 0.05), key=lambda t: _seg_f1(clf, train,
float(t)))
130         v = data[test]
131         probs = clf.predict_proba(v["X"])
132         auc = roc_auc(v["y"], probs)
133         segs = probs_to_segments(probs, v["times"], float(thr))
134         sc = score(segs, v["gts"], 0.5)
135         # Oracle threshold (fit-on-TEST, upper bound): isolates the representation
ceiling
136         # from the threshold-transfer gap. Optimistic ? for diagnosis only.
137         othr = max(np.arange(0.2, 0.81, 0.05),
138                   key=lambda t: score(probs_to_segments(probs, v["times"], float(t)),
v["gts"], 0.5).f1)
139         osc = score(probs_to_segments(probs, v["times"], float(othr)), v["gts"], 0.5)
140         results.append({"video": test, "thr": float(thr), "auc": auc,
141                       "f1": sc.f1, "precision": sc.precision, "recall": sc.recall,
142                       "n_pred": len(segs), "n_gt": len(v["gts"]),
143                       "oracle_thr": float(othr), "oracle_f1": osc.f1})
144         print(f"[held-out {test}] per-frame AUC {auc:.3f} | LOVO-threshold F1
{sc.f1:.3f} "
145               f"(P {sc.precision:.3f} R {sc.recall:.3f}, thr {thr:.2f} from train) | "
146               f"oracle-threshold F1 {osc.f1:.3f} (thr {othr:.2f}, fit-on-test upper
bound)")
147         mean_f1 = sum(r["f1"] for r in results) / len(results)
148         print(f"\nlearned LOVO mean held-out F1: {mean_f1:.3f}")
149         return {"per_video": results, "mean_f1": mean_f1}
150
151
152     def main() -> None:
153         ap = argparse.ArgumentParser(description=__doc__)
154         ap.add_argument("--manifest", required=True, help="JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
155         args = ap.parse_args()
156         with open(args.manifest, encoding="utf-8") as f:
157             run(json.load(f))
158
159
160     if __name__ == "__main__":

```

31. assistant (2026-06-18T07:49:17.026Z)

Important finding here. In `rally\_seq\_proto.run` (line 147), `mean\_f1 = sum(...) / len(results)` will raise `ZeroDivisionError` if `specs` is empty. But more relevant to the harness: `gen0/harness.run` calls `lovo\_run(specs)` then `train\_final(specs)`. In `train\_final`, `np.vstack([v["X"] for v in vids])` would raise on empty list. These are edge cases requiring an empty manifest ? an operational error rather than a defect. Let me note but not over-weight.

A more interesting question: `gate\_verdict`'s `sign\_test`. Let me re-examine the `p` calculation:

`p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 \*\* n)` ? this is the one-sided p-value for the sign test. That's correct: $P(X \geq \text{wins})$ under $\text{Binomial}(n, 0.5)$. Looks correct.

Now let me check the `gen0` harness `gate\_verdict` floor comparison line 113: `floor\_mean = float(floor["mean\_f1"])`. The floor JSON is `eval\_baselines/heuristic\_lovo\_n6.json`. Let me also check the paired list at line 118: `per = floor.get("per\_video", {})` then `paired = [(r["f1"], per[r["video"]]) for r in eval\_res["per\_video"] if r["video"] in per]`. Here `per[r["video"]]?` if `per` is a dict mapping video->f1 (a float), this works. But the floor `per\_video` shape? let me verify it's a dict of video->float, not a list. Let me check the test and the baseline file.

32. user (2026-06-18T07:49:17.532Z)

```
1      """Gen-0 harness (training/gen0): end-to-end on SYNTHETIC fixtures only.
2
3      Real trajectories/labels are never committed (training/README.md rule 4 ? consent ledger
4      pending). The synthetic corpus mirrors tests/test_rally_seq_proto.py: separable rally vs
5      dead-time signal, three rallies per 'video'.
6      """
7      import csv
8      import json
9
10     import numpy as np
11     import pytest
12
13     from training.gen0 import harness
14
15
16     def _write_traj(path, fps=10.0):
17         rows = [("Frame", "Visibility", "X", "Y")]
18         n = int(120 * fps)
19         for f in range(n):
20             t = f / fps
21             in_rally = (t < 20) or (40 <= t < 60) or (80 <= t < 100)
22             if in_rally:
23                 x = 50 + 45 * np.sin(2 * np.pi * (f % 10) / 10.0)
24                 rows.append((f, 1, round(float(x), 2), 50))
25             else:
26                 rows.append((f, 1, 50.0, 50) if f % 5 == 0 else (f, 0, -1, -1))
27         with open(path, "w", newline="") as fh:
28             csv.writer(fh).writerows(rows)
29         return str(path)
30
31
32     def _write_labels(path):
33         with open(path, "w", newline="", encoding="utf-8") as fh:
34             w = csv.writer(fh)
35             w.writerow(["rally_number", "start_time", "end_time"])
36             w.writerow([1, 0.0, 20.0])
37             w.writerow([2, 40.0, 60.0])
38             w.writerow([3, 80.0, 100.0])
39         return str(path)
40
41
42     @pytest.fixture
43     def corpus(tmp_path):
44         specs = [{
45             "name": f"v{i}", "trajectory": _write_traj(tmp_path / f"t{i}.csv"),
46             "fps": 10.0, "frame_width": 100.0, "labels": _write_labels(tmp_path /
47 f"g{i}.csv"),
48         } for i in range(2)]
49         mpath = tmp_path / "manifest.json"
50         mpath.write_text(json.dumps(specs), encoding="utf-8")
51         return str(mpath), tmp_path
52
53     def test_harness_end_to_end_emits_artifact_bundle(corpus):
54         mpath, tmp_path = corpus
```

```

55     floor = tmp_path / "floor.json"
56     floor.write_text(json.dumps({
57         "name": "synthetic-floor", "mean_f1": 0.10,
58         "per_video": {"v0": 0.10, "v1": 0.05},
59     }), encoding="utf-8")
60
61     out = harness.run(mpath, str(tmp_path / "artifacts"), "0.0.1",
62 floor_path=str(floor))
63
64     bundle = tmp_path / "artifacts" / "0.0.1"
65     assert (bundle / "weights.json").exists()
66     assert (bundle / "manifest.json").exists()
67
68     # Manifest contract (the reviewable surface).
69     m = json.loads((bundle / "manifest.json").read_text(encoding="utf-8"))
70     assert m["artifact"]["name"] == "rally_tal_gen0"
71     assert m["artifact"]["weights_sha256"]
72     assert m["feature_schema"]["version"] >= 1
73     assert m["feature_schema"]["fps_semantics"] == "per-second"
74     assert 0.2 <= m["calibration"]["threshold"] <= 0.8
75     assert len(m["training_lineage"]["videos"]) == 2
76     assert all(v["gt_hash"] for v in m["training_lineage"]["videos"])
77     assert m["eval"]["mean_f1"] > 0.5 # separable synthetic signal
78     assert m["attestation"]["no_paid_llm_training_labels"] is True
79
80     # Gate: floor beaten, but NEVER self-ships (provisional target + C3 open; see #172).
81     g = m["gate"]
82     assert g["ship"] is False
83     assert g["floor_passed"] is True
84     assert any("C3" in b for b in g["blockers"])
85     assert any("NOT YET CALIBRATED" in b for b in g["blockers"])
86     assert any("#172" in b for b in g["blockers"])
87     assert out["gate"] == g
88
89     def test_weights_round_trip_and_schema_pin(corpus):
90         mpath, tmp_path = corpus
91         harness.run(mpath, str(tmp_path / "artifacts"), "0.0.2", floor_path=None)
92         payload = json.loads((tmp_path / "artifacts" / "0.0.2" /
93 "weights.json").read_text())
94         from backend.eval.classifier import LogisticRegression
95         from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA_VERSION
96         clf = LogisticRegression.from_dict(payload)
97         assert payload["feature_schema_version"] == FEATURE_SCHEMA_VERSION
98         assert list(clf.feature_names) == FEAT_NAMES
99         probs = clf.predict_proba(np.zeros((4, len(FEAT_NAMES))))
100         assert probs.shape == (4,)
101
102     def test_sign_test_math():
103         # 5/6 wins -> one-sided p = 7/64 ~ 0.109 (NOT significant ? honesty pinned).
104         wins, n, p = harness.sign_test([1, 1, 1, 1, 1, 0], [0, 0, 0, 0, 0, 1])
105         assert (wins, n) == (5, 6)
106         assert p == pytest.approx(7 / 64)
107         # 6/6 -> p = 1/64 < 0.05.
108         _, _, p6 = harness.sign_test([1] * 6, [0] * 6)
109         assert p6 == pytest.approx(1 / 64)
110

```

33. assistant (2026-06-18T07:49:32.530Z)

The floor `per\_video` is a dict `{video: f1}` ? confirmed by the test. So `per[r["video"]` returns a float. Correct.

Now let me reconsider the `lovo\_run` in `rally\_seq\_proto.run` vs `train\_final` use **different

GT loaders**: `run` uses `load\_golden\_gts(s["labels"])` (line 115) while `train\_final` uses `load\_gt\_intervals(s["labels"], strict=False)` (line 88). That's a potential inconsistency but both are existing-code design choices ? not necessarily a bug since the threshold from `train\_final` is recorded with the note that LOVO is the honest number.

Let me now zero in on something concrete in `nightly\_regression.py`. Look at the `compare\_all` and the documented exit code 4 (STALE). But there's a subtle issue: a NEW config present in `current` but not in `baseline` is marked `stale` (line 353). That's handled. But what about the ****reverse****: a config in baseline but removed from `current`? `compare\_all` only iterates `cur\_cfgs`, so a removed config silently disappears ? no stale flag. Minor; the tracked configs are hardcoded so this won't happen in practice.

Let me check a more impactful issue. The `format\_report` uses `current.get('git\_commit')` but in `main`, `snapshot["git\_commit"]` is set at line 586 AND `save\_baseline` also sets it. But in the comparison path (non-update), `snapshot["git\_commit"]` is set at 586. Good.

Let me now carefully inspect `run\_process\_and\_stitch.py` for the `output\_dir` vs `cfg.output.output\_dir` mismatch. Line 26: `output\_dir = r"C:\...\Testing"` (hardcoded). Line 39: `os.makedirs(cfg.output.output\_dir, ...)` ? creates the config output dir. Line 41: `db\_path = os.path.join(cfg.output.output\_dir, ...)` . Line 150: `VideoCompiler(output\_dir, ...)` ? uses the hardcoded `output\_dir`. This is a personal driver script with hardcoded paths, by design. Not a defect to report.

Let me look carefully at one thing in `export.py` `\_fmt\_hms` and `format\_rally\_summary` ? the `total\_dur` uses `end\_time - start\_time` fallback but `s.get("duration", ...)` . If `duration` key is present but `None`... `s.get("duration", fallback)` returns `None` if the key exists with value None. Then `float(None)` raises TypeError. Let me check if that's reachable. Same pattern in `basic\_stats` line 96 and `\_highlights`. This is a real fragility but requires `duration` key present with `None` value. Let me check whether segments can have `duration: None`.

34. user (2026-06-18T07:49:34.568Z)

Found 10 files
backend\tools\export.py
backend\reporting\harvest.py
backend\pipeline\validators\format.py
backend\pipeline\segmenters\yolo_hybrid.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\motion.py
backend\pipeline\segmenters\gemini.py
backend\eval\experiment.py
backend\eval\export_wasb_segments.py
backend\eval\audio\e1_probe.py

35. assistant (2026-06-18T07:49:46.626Z)

The `duration: None` case is theoretical ? segments always set duration to a float. Not worth reporting as a defect.

Let me now look at the `harvest.py` `\_highlights` coverage calc and `video\_meta\_from\_context` more carefully. Line 60: `if fmt.get("duration")`. Then line 65: `if fmt.get("format\_name")`: `meta["container"] = fmt.get("format\_name")`. Note this OVERWRITES the container set at line 59 from the stream codec. The line 59 logic sets container to `fmt.get("format\_name")` or `s.get("codec\_name")`. Then line 65-66 overwrites with `fmt.get("format\_name")` ? but only if it's truthy. So if `format\_name` is set, container = format_name in both. If format_name is None but codec_name set, line 59 sets container to codec_name, and line 65 condition false, so it stays codec_name. Consistent. Not a bug.

Let me re-examine the `r\_frame\_rate` parsing at lines 51-56. `num, den = (int(x) for x in fps\_str.split("/"))`. If `fps\_str` is `"30000/1001"` ? fine. If `fps\_str` has no `/` (e.g. `"30"`), `split("/")` returns `["30"]`, unpacking into `num, den` raises `ValueError` ? caught. If `fps\_str` is `"30/1/2"` ? 3 elements, unpack raises ValueError ? caught. Good defensive handling.

Now, an important one: line 52 `int(x)` ? if `r\_frame\_rate` is `"24000/1001"`, fine. But what if

it's a float like `"23.976"`? Then there's no `/'`, split gives one element, ValueError caught. OK.

Let me now look at the doctor.py more carefully ? it always runs `run\_pip\_install` at the end regardless of diagnostics. Line 122-124: `init\_default\_config()`, `run\_diagnostics()`, `run\_pip\_install()`. The exit code is never set ? `\_\_main\_\_` block doesn't `sys.exit`. So even if FFmpeg fails, exit code is 0. Is that a defect? The script is a bootstrap/setup helper; the diagnostics print failures but the script's job is to install. Returning 0 always is arguably acceptable for a setup script but it does mean a CI or wrapper can't detect failure. Given the AREA mentions "exit codes", let me note this as low-confidence ? it's a known characteristic, not clearly a defect.

Let me look at the `nightly\_regression.ps1` since it's referenced and in scope.

36. user (2026-06-18T07:49:47.568Z)

```
1      <#
2      .SYNOPSIS
3          Nightly rally-quality regression tripwire ? wrapper for the Windows Scheduled Task.
4
5      .DESCRIPTION
6          Re-scores the golden corpus under the current checkout via
scripts/nightly_regression.py
7          (CPU, seconds ? the WASB trajectories are CACHED) and diffs vs the frozen baseline.
Writes a
8          dated report + log under output/nightly_regression/ and a LATEST_STATUS.txt marker,
then exits
9          with the checker's exit code so Task Scheduler records PASS / REGRESSION / STALE.
10
11         GRACEFUL no-op (exit 0) when the checker or the local golden corpus isn't present
yet (e.g.
12         before this PR is merged into the checkout the task points at) ? so a registered
task never
13         errors in the interim and self-activates once the files land.
14
15         Cloud agents can't do this: the cached trajectories + golden features live OUTSIDE
the repo on
16         this machine. Hence a LOCAL scheduled task, not a CronCreate/cloud routine.
17
18     .PARAMETER RepoRoot
19         Repo root to test. Default: the parent of this script's directory.
20
21     .PARAMETER PythonExe
22         Python interpreter. Default: 'python'.
23
24     .PARAMETER Pull
25         If set, 'git pull --ff-only' before scoring (off by default ? never auto-touch a
shared checkout
26         that may hold a sibling session's WIP).
27     #>
28     [CmdletBinding()]
29     param(
30         [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
31         [string]$PythonExe = 'python',
32         [switch]$Pull
33     )
34
35     $ErrorActionPreference = 'Continue'
36     $date = Get-Date -Format 'yyyy-MM-dd'
37     $reportDir = Join-Path $RepoRoot 'output\nightly_regression'
38     $logDir = Join-Path $reportDir 'logs'
39     New-Item -ItemType Directory -Force -Path $logDir | Out-Null
40     $log = Join-Path $logDir "$date.log"
41     $statusFile = Join-Path $reportDir 'LATEST_STATUS.txt'
```

```

42
43     function Write-Status([string]$status, [int]$code, [string]$detail) {
44         $stamp = Get-Date -Format 'yyyy-MM-dd HH:mm:ss'
45         "$stamp status=$status exit=$code $detail" | Out-File -FilePath $statusFile
46         -Encoding utf8
47     }
48
49     $checker = Join-Path $RepoRoot 'scripts\nightly_regression.py'
50     $manifest = Join-Path $RepoRoot 'output\human_lovo_manifest.json'
51
52     # Graceful no-op until the checker + corpus are present (pre-merge / corpus absent).
53     if (-not (Test-Path $checker)) {
54         "$date] nightly_regression.py not present at $checker ? skipping (awaiting merge)."
55         | Tee-Object -FilePath $log
56         Write-Status 'SKIPPED' 0 'checker absent (awaiting merge)'
57         exit 0
58     }
59     if (-not (Test-Path $manifest)) {
60         "$date] golden manifest not present at $manifest ? skipping (corpus absent on this
61         machine)." | Tee-Object -FilePath $log
62         Write-Status 'SKIPPED' 0 'golden corpus absent'
63         exit 0
64     }
65
66     Set-Location $RepoRoot
67     if ($Pull) {
68         "$date] git pull --ff-only" | Tee-Object -FilePath $log -Append
69         & git -C $RepoRoot pull --ff-only 2>&1 | Tee-Object -FilePath $log -Append
70     }
71
72     $head = (& git -C $RepoRoot rev-parse --short HEAD 2>$null)
73     "$date] nightly rally-quality regression ? HEAD $head" | Tee-Object -FilePath $log
74     -Append
75
76     & $PythonExe $checker 2>&1 | Tee-Object -FilePath $log -Append
77     $code = $LASTEXITCODE
78
79     switch ($code) {
80         0 { Write-Status 'PASS' 0 "report: $reportDir\$date.md" }
81         1 { Write-Status 'REGRESSION' 1 "report: $reportDir\$date.md"
82           "$date] *** NIGHTLY REGRESSION DETECTED ? see $reportDir\$date.md ***" | Tee-
83           Object -FilePath $log -Append }
84         4 { Write-Status 'STALE' 4 "baseline refresh needed ? see $reportDir\$date.md" }
85         default { Write-Status 'ERROR' $code "checker error ? see $log" }
86     }
87
88     exit $code
89
90

```

37. user (2026-06-18T07:49:48.136Z)

```

1     <#
2     .SYNOPSIS
3         Register (or remove) the nightly rally-quality regression as a Windows Scheduled
4         Task.
5
6     .DESCRIPTION
7         Creates a daily user-level task that runs scripts/nightly_regression.ps1 against a
8         repo
9         checkout. The action is inline-guarded, so it's safe to register BEFORE this work is
10        merged
11        into the target checkout: it no-ops + logs "awaiting" until the wrapper exists
12        there, then
13        self-activates. Idempotent (-Force re-registers); reversible (-Unregister).
14

```



```

11         Registers under the current user (no admin / no -RunLevel Highest needed ?
consistent with the
12         repo's other scheduled tasks). Uses -StartWhenAvailable so a run missed while the
machine slept
13         is caught up on wake.
14
15     .EXAMPLE
16         # Register against the main checkout, nightly at 03:00:
17         pwsh -File scripts/register_nightly_task.ps1 -RepoRoot
'C:\Users\avidu\Projects\badminton-highlight-indexer'
18
19     .EXAMPLE
20         # Remove it:
21         pwsh -File scripts/register_nightly_task.ps1 -Unregister
22     #>
23     [CmdletBinding()]
24     param(
25         [string]$RepoRoot = (Split-Path -Parent (Split-Path -Parent
$MyInvocation.MyCommand.Path)),
26         [string]$TaskName = 'KhelSutraNightlyRegression',
27         [string]$Time = '03:00',
28         [switch]$Pull,
29         [switch]$Unregister
30     )
31
32     if ($Unregister) {
33         Unregister-ScheduledTask -TaskName $TaskName -Confirm:$false -ErrorAction
SilentlyContinue
34         Write-Host "Unregistered scheduled task '$TaskName'."
35         return
36     }
37
38     $wrapper = Join-Path $RepoRoot 'scripts\nightly_regression.ps1'
39     $ps = (Get-Command pwsh.exe -ErrorAction SilentlyContinue).Source
40     if (-not $ps) { $ps = 'powershell.exe' }
41
42     # Inline-guarded action: call the wrapper if present, else log "awaiting merge" and exit
0.
43     $marker = Join-Path $RepoRoot 'output\nightly_regression\LATEST_STATUS.txt'
44     $pullArg = if ($Pull) { ' -Pull' } else { '' }
45     $inner = @"
46     `$w = '$wrapper'
47     if (Test-Path `$w) { & `$w -RepoRoot '$RepoRoot'$pullArg }
48     else {
49         New-Item -ItemType Directory -Force -Path (Split-Path -Parent '$marker') | Out-Null
50         "$$(Get-Date -Format 'yyyy-MM-dd HH:mm:ss') status=SKIPPED exit=0 wrapper absent
(awaiting merge) at '$wrapper'" | Out-File -FilePath '$marker' -Encoding utf8
51     }
52     "@
53     $encoded = [Convert]::ToBase64String([Text.Encoding]::Unicode.GetBytes($inner))
54
55     $action = New-ScheduledTaskAction -Execute $ps `
56         -Argument "-NoProfile -ExecutionPolicy Bypass -EncodedCommand $encoded"
57     $trigger = New-ScheduledTaskTrigger -Daily -At $Time
58     $settings = New-ScheduledTaskSettingsSet -StartWhenAvailable -DontStopOnIdleEnd `
59         -AllowStartIfOnBatteries -DontStopIfGoingOnBatteries `
60         -ExecutionTimeLimit (New-TimeSpan -Minutes 30)
61     $principal = New-ScheduledTaskPrincipal -UserId "$env:USERDOMAIN\$env:USERNAME"
-LogonType Interactive -RunLevel Limited
62
63     Register-ScheduledTask -TaskName $TaskName -Action $action -Trigger $trigger `
64         -Settings $settings -Principal $principal -Force `
65         -Description "Nightly rally-quality regression tripwire: re-scores the golden corpus
(cached trajectories, CPU) vs eval_baselines/nightly_baseline.json. Writes
output/nightly_regression/<date>.md. Exit 1=regression, 4=stale baseline." | Out-Null

```

```

66
67     Write-Host "Registered scheduled task '$TaskName' ? daily at $Time, RepoRoot=$RepoRoot."
68     Write-Host "Wrapper: $wrapper"
69     Write-Host "Run now to test:   Start-ScheduledTask -TaskName '$TaskName'"
70     Write-Host "Remove:           pwsh -File scripts/register_nightly_task.ps1 -Unregister"
71

```

38. assistant (2026-06-18T07:49:58.897Z)

The PS1 wrapper looks careful. One observation: the `nightly\_regression.ps1` switch on `\$code` doesn't handle exit codes 2 (operational error) or 3 (no baseline) explicitly ? they fall to `default ? ERROR`. That's actually correct behavior (treats them as errors). Fine.

Now let me carefully reconsider the `nightly\_regression.py` exit code path. There's something important. Look at `main`: when `--no-report` is NOT set and there's a regression (code 1), the report is written, then `print(report)`, then logs the error and returns code. The exit code path looks correct.

Let me reconsider one thing about the `summarize\_run` function and the `merge\_rate` gating. `GATED\_LOWER = ("merge\_rate",)`. In `compare\_config`, the `merge\_rate` aggregate is gated. But `merge\_rate` must be in `AGG\_SNAPSHOT` ? yes it is (line 91). Good.

But wait ? let me check the per-video guard metrics `PER\_VIDEO\_GUARD = ("split\_count", "merge\_count")` and verify those are in `PV\_SNAPSHOT` (line 94-96): `split\_count`, `merge\_count` ? yes. Good.

Now let me check: in `compare\_config`, the per-video guard runs `for vid in sorted(base\_names & cur\_names)`. For each, `b, c = base\_pv[vid].get(m), cur\_pv[vid].get(m)`. If a metric is missing (None), skip. If `c > b + tol`, regression. The `over\_seg\_tol=0` makes this strict. Now, the comparison `c > b + tols.over\_seg\_tol`: b and c are ints (from JSON they'd be ints or floats). Fine.

Let me now verify a genuine concern: does `rally\_seg\_eval.evaluate` accept `min\_crossings`? Let me check the signature, since `run\_corpus` calls `rally\_seg\_eval.evaluate(golden, cfg.preset, iou=iou, min\_crossings=cfg.min\_crossings)`.

39. user (2026-06-18T07:50:00.732Z)

```

101: def load_golden(manifest_path: str = DEFAULT_MANIFEST,
147: def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,

```

40. user (2026-06-18T07:50:04.401Z)

```

101     def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102                     features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103         """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
rally
104         intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept but
flagged."""
105         with open(manifest_path) as fh:
106             manifest = json.load(fh)
107             out: List[Dict[str, Any]] = []
108             for v in manifest:
109                 feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
110                 gts: List[Interval] = []
111                 if os.path.exists(feat):
112                     with open(feat) as fh:
113                         gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
[])]
114                 out.append({"name": v["name"], "trajectory": v["trajectory"],
115                           "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
"gts": gts})
116             return out
117

```

```

118
119     def windows_for(video: Dict[str, Any], preset: WindowingPreset,
120                     min_crossings: int = 0) -> List[Interval]:
121         """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
122
123         ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
124         ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
125         times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
126         from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
127         windowing (W1) leaves behind.
128
129         NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
130         drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
131         strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
132         """
133         from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
134         points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])
135         wins = windowing.build_windows(points, video["fps"], video["frame_width"], preset)
136         ivs = windowing.windows_to_intervals(wins)
137         if min_crossings > 0 and ivs:
138             from backend.pipeline.detectors.rally_gate import gate_windows
139             gated = [(g.start, g.end)
140                     for g in gate_windows(points, ivs, video["fps"],
min_crossings=min_crossings)
141                     if g.kept]
142             # Gating emptied a non-empty video ? keep the ungated windows.
143             ivs = gated if gated else ivs
144         return ivs
145
146
147     def evaluate(golden: List[Dict[str, Any]], preset: WindowingPreset,
148                 iou: float = DEFAULT_IOU, min_crossings: int = 0,
149                 tau_start: Optional[float] = None, tau_end: Optional[float] = None) ->
Dict[str, Any]:
150         """Score every golden video under ``preset`` (+ optional net-crossings gate); return
per-video
151         rows + the aggregate. Each row carries both the tIoU set and the R2 tolerance-match
block."""
152         rows: List[Dict[str, Any]] = []
153         for v in golden:
154             if not v["gts"] or not os.path.exists(v["trajectory"]):
155                 continue
156             preds = windows_for(v, preset, min_crossings)
157             row = score_intervals(preds, v["gts"], iou, tau_start, tau_end)
158             row["name"] = v["name"]
159             rows.append(row)
160         return {"preset": preset.to_dict(), "iou": iou, "min_crossings": min_crossings,
161               "rows": rows, "aggregate": aggregate(rows)}
162
163
164     def compare(golden: List[Dict[str, Any]], base: WindowingPreset, cand: WindowingPreset,
165                 iou: float = DEFAULT_IOU, base_min_crossings: int = 0,
166                 cand_min_crossings: int = 0,
167                 tau_start: Optional[float] = None, tau_end: Optional[float] = None,
168                 guard_weight_merge: Optional[float] = None,
169                 guard_weight_split: Optional[float] = None,
170                 guard_use_rates: bool = True) -> Dict[str, Any]:
171         """Baseline vs candidate (windowing and/or net-crossings gate): per-video paired ?

```

```

(+ CI +
172     sign-test) on both tIoU-F1 and the R2 ``tol_f1`` ? the per-tier 'number'. Also
reports ? on the
173     over-segmentation metrics AND the **R1 net over-seg promotion guard**
(``over_seg_guard``):
174     the honest promote/block verdict for cand-over-base, with the strict per-video rule
reported
175     alongside so the refinement is auditable.""
176     from backend.eval import tolerance_metrics as tm
177     b = evaluate(golden, base, iou, base_min_crossings, tau_start, tau_end)
178     c = evaluate(golden, cand, iou, cand_min_crossings, tau_start, tau_end)
179     by_name_b = {r["name"]: r for r in b["rows"]}
180     paired = [(by_name_b[r["name"]], r) for r in c["rows"] if r["name"] in by_name_b]
181     delta: Dict[str, Any] = {}
182     for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate",
"segment_count_ratio"):
183         delta[k] = eval_stats.paired_delta_ci([rb[k] for rb, _ in paired],
184                                             [rc[k] for _, rc in paired])
185     gkw: Dict[str, Any] = {"use_rates": guard_use_rates}
186     if guard_weight_merge is not None:
187         gkw["weight_merge"] = guard_weight_merge
188     if guard_weight_split is not None:
189         gkw["weight_split"] = guard_weight_split
190     guard = tm.over_seg_guard([rb for rb, _ in paired], [rc for _, rc in paired], **gkw)

```

41. assistant (2026-06-18T07:50:11.124Z)

Notice in `evaluate` (line 154): videos with no GTs **or** missing trajectory are skipped (`continue`). This means they never appear in `rows`, hence never in `per_video`. The nightly's `run_corpus` then computes `skipped[cfg.name] = [n for n in manifest_names if n not in scored]`. Good ? skipped videos are surfaced.

Now here's a subtle but real point about the `aggregate`n``. Let me check the `score_intervals`keys` to confirm tol_f1, f1@0.5, f1@0.25, merge_rate, split_count, merge_count all exist in a row, otherwise summarize_run`'s per_video` dict comprehension {k: r[k] for k in PV_SNAPSHOT if k in r} silently drops missing keys ? and the gate then can't compare. That's defensive if k in r. Let me verify f1@0.5` vs f1` naming in score_intervals.`

42. user (2026-06-18T07:50:13.392Z)

```

67:     "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
69:     "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
74:     "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
77:     "split_count": osr["split_count"], "merge_count": osr["merge_count"],
81: _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate", "segment_count_ratio",
83:             "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median", "dend_abs_median")
182:     for k in ("f1", "f1@0.25", "tol_f1", "merge_rate", "split_rate", "segment_count_ratio"):
236:         lines.append(f"| {r['name']} | {r['f1']:.3f} | {r['f1@0.25']:.3f} |
{r['f1@0.5']:.3f} | "
237:                     f"{int(r['merges'])} | {r['merge_rate']:.2f} |
{r['segment_count_ratio']:.2f} | "
241:                     f"F1 {_ci(a['f1'])} · F1@0.25 {_ci(a['f1@0.25'])} · F1@0.5 {_ci(a['f1@0.5'])}
. "
242:                     f"merge_rate {_ci(a['merge_rate'])} · seg_ratio
{_ci(a['segment_count_ratio'])}")
245:     if rows and "tol_f1" in rows[0]:
251:         lines.append(f"| {r['name']} | {r['tol_f1']:.3f} | {r['tol_precision']:.2f} | "
253:                     f"{r['dend_abs_median']:.2f} | {int(r['split_count'])} |
{int(r['merge_count'])} |")
254:         lines += [f"", f"**R2 aggregate (n={a['n']}):** tolF1 {_ci(a['tol_f1'])} · "

```

43. assistant (2026-06-18T07:50:18.833Z)

Now let me verify the aggregate keys exist in `aggregate()`. The `AGG_SNAPSHOT` includes`

`segment\_count\_ratio` but `\_AGG\_KEYS` uses `segment\_count\_ratio` too. Let me check that `aggregate()` produces dict-shaped entries with `mean`. The `summarize\_run` line 181: `agg[k]["mean"] if isinstance(agg.get(k), dict) else None`. So if a key isn't a dict, it becomes None. Let me check the aggregate structure to confirm `segment\_count\_ratio`, `dstart\_abs\_median`, `dend\_abs\_median`, `num\_pred`, `num\_gt` keys.

`PV\_SNAPSHOT` includes `num\_pred`, `num\_gt` (line 96). But the row keys from `score\_intervals` ? does it produce `num\_pred`/`num\_gt`? Let me check.

44. user (2026-06-18T07:50:19.936Z)

```

55         from backend.eval import tolerance_metrics as tm
56         ts = tm.TAU_START if tau_start is None else tau_start
57         te = tm.TAU_END if tau_end is None else tau_end
58         sc = metrics.score(preds, gts, iou_threshold=iou)
59         f1k = segmentation_metrics.f1_at_overlaps(preds, gts)
60         ms = segmentation_metrics.merge_split_report(preds, gts)
61         tol_matches, tol_p, tol_r, tol_f1 = tm.match_1to1(preds, gts, ts, te)
62         be = tm.boundary_error_report(preds, gts) # unconditional best-overlap (the
R4-aiming diagnostic)
63         osr = tm.over_seg_report(preds, gts)
64         return {
65             "f1": sc.f1, "precision": sc.precision, "recall": sc.recall, "mean_iou":
sc.mean_iou,
66             "mean_start_error": sc.mean_start_error, "mean_end_error": sc.mean_end_error,
67             "f1@0.1": f1k[0.1], "f1@0.25": f1k[0.25], "f1@0.5": f1k[0.5],
68             "segment_count_ratio": segmentation_metrics.segment_count_ratio(preds, gts),
69             "merge_rate": ms["merge_rate"], "split_rate": ms["split_rate"],
70             "merges": ms["merges"], "splits": ms["splits"], "missed": ms["missed"],
71             "spurious": ms["spurious"], "num_pred": len(preds), "num_gt": len(gts),
72             # --- R2 (tolerance-match) ? the task-faithful headline block ---
73             "tau_start": ts, "tau_end": te,
74             "tol_f1": tol_f1, "tol_precision": tol_p, "tol_recall": tol_r,
75             "tol_matched": float(len(tol_matches)),
76             "dstart_abs_median": be["dstart"]["abs_median"], "dend_abs_median":
be["dend"]["abs_median"],
77             "split_count": osr["split_count"], "merge_count": osr["merge_count"],
78         }
79
80
81         _AGG_KEYS = ("f1", "f1@0.25", "f1@0.5", "merge_rate", "split_rate",
"segment_count_ratio",
82                     "precision", "recall",
83                     "tol_f1", "tol_precision", "tol_recall", "dstart_abs_median",
"dend_abs_median")
84
85
86         def aggregate(rows: List[Dict[str, Any]]) -> Dict[str, Any]:
87             """Aggregate per-video rows: mean (+ bootstrap 95% CI) of the headline metrics
across videos.
88
89             Per-video means (not pooled) keep videos the unit of evidence, matching the
harness's
90             leave-one-video-out discipline (windows within a video are correlated)."""
91             out: Dict[str, Any] = {"n": len(rows)}
92             for k in _AGG_KEYS:
93                 vals = [r[k] for r in rows]
94                 out[k] = eval_stats.mean_with_ci(vals) if vals else None
95             return out
96
97
98         # ----- #
99         # Golden corpus I/O
100        # ----- #

```

45. assistant (2026-06-18T07:50:31.620Z)

Now I can see a real discrepancy. `AGG\_SNAPSHOT` in `nightly_regression.py` (line 89-92) includes `split\_rate`:

```
...
AGG_SNAPSHOT = ("tol_f1", "tol_precision", "tol_recall", "f1@0.5", "f1@0.25", "precision",
"recall",
    "merge_rate", "split_rate", "segment_count_ratio", "dstart_abs_median", "dend_abs_median")
...
```

`\_AGG\_KEYS` (line 81-83) includes `split\_rate` and `segment\_count\_ratio` ? both present in aggregate. Good. So all AGG_SNAPSHOT keys are in aggregate output. `summarize\_run` extracts `[\"mean\"]` from each. All fine.

So the metrics plumbing is consistent. No bug there.

Let me now think about the `**net-crossings gate**` `windows\_for`: when `min\_crossings=0`, gate is skipped. All tracked nightly configs have `min\_crossings=0`. Fine.

Let me reconsider the `\_preset\_comparable` function. It compares preset dicts excluding `name`. But the baseline stores `preset` as a dict (from `preset.to\_dict()`). The current snapshot also stores `preset.to\_dict()`. The comparison excludes `name`. For the three configs, the presets differ (e.g. `milestone` has `name="milestone"` and different `max\_window\_duration`). Since `compare\_all` matches by config NAME (`default`, `milestone`, `baseline\_off`), and compares preset dicts within the same config name. Good.

But wait ? there's a subtle issue worth checking. `replace(SERVED, name="milestone", ...)` . The `WindowingPreset.to\_dict()` includes the `name` field. The baseline stores `default`'s preset which is `SERVED` (with `name="served"` presumably). If a future change to `SERVED`'s knobs happens, `\_preset\_comparable` detects it (excluding name) ? stale. Good design.

I've done a thorough pass. Let me now reconsider the one genuinely interesting candidate: in `run\_phase3\_eval.py`, line 89 `entries = manifest.get("eval\_sets", [])`. Then line 111 `e["video\_id"]`, line 124 `e["csv"]`. If `eval\_sets` is empty, `targets` is empty, loop doesn't run, `agg = batch.aggregate([], stages)`. Let me verify `batch.aggregate` handles empty.

46. user (2026-06-18T07:50:33.646Z)

```
22:def resolve_video_path(local_path: Optional[str], videos_root: str) -> Optional[str]:
43:def default_stages_for(segmenter: str, requested: str = "auto") -> List[str]:
64:def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
103:def format_aggregate(agg: Dict[str, dict]) -> str:
```

47. user (2026-06-18T07:50:36.394Z)

```
64     def aggregate(per_video: List[dict], stages: List[str]) -> Dict[str, dict]:
65         """Aggregate per-video stage scores into corpus-level metrics.
66
67         `per_video` is a list of dicts shaped like ``report_to_dict`` output (each has
68         ``stages[stage][\"score\"]`` with tp/fp/fn/mean_iou/...). For each stage we
69         compute **micro** precision/recall/F1 (pooling TP/FP/FN across all videos) and
70         a TP-weighted mean IoU, plus the macro mean of per-video F1.
71         """
72         out: Dict[str, dict] = {}
73         for stage in stages:
74             tp = fp = fn = 0
75             iou_weighted = 0.0
76             f1s: List[float] = []
77             n_videos = 0
78             for rep in per_video:
79                 sr = rep.get("stages", {}).get(stage)
80                 if not sr:
81                     continue
82                 s = sr["score"]
```

```

83         tp += s["true_positives"]
84         fp += s["false_positives"]
85         fn += s["false_negatives"]
86         iou_weighted += s["mean_iou"] * s["true_positives"]
87         f1s.append(s["f1"])
88         n_videos += 1
89     precision = _safe_div(tp, tp + fp)
90     recall = _safe_div(tp, tp + fn)
91     out[stage] = {
92         "videos": n_videos,
93         "tp": tp, "fp": fp, "fn": fn,
94         "micro_precision": precision,
95         "micro_recall": recall,
96         "micro_f1": _safe_div(2 * precision * recall, precision + recall),
97         "macro_f1": _safe_div(sum(f1s), len(f1s)),
98         "mean_iou": _safe_div(iou_weighted, tp),
99     }
100     return out
101
102
103 def format_aggregate(agg: Dict[str, dict]) -> str:
104     """Human-readable corpus-level summary table."""
105     lines = [
106         "=" * 78,
107         " CORPUS AGGREGATE",
108         "=" * 78,
109         f" {'Stage':<12} {'Vids':>4} {'TP':>5} {'FP':>5} {'FN':>5} "
110         f"{'mPrec':>6} {'mRec':>6} {'microF1':>8} {'macroF1':>8} {'mIoU':>6}",
111         "-" * 78,
112     ]
113     for stage, a in agg.items():
114         lines.append(
115             f" {stage:<12} {a['videos']:>4} {a['tp']:>5} {a['fp']:>5} {a['fn']:>5} "
116             f"{a['micro_precision']:>6.2f} {a['micro_recall']:>6.2f} "
117             f"{a['micro_f1']:>8.2f} {a['macro_f1']:>8.2f} {a['mean_iou']:>6.2f}"
118         )
119     lines.append("=" * 78)
120     return "\n".join(lines)
121

```

48. assistant (2026-06-18T07:50:47.615Z)

`aggregate` uses `\_safe\_div`, so empty is handled. `format\_aggregate` iterates `agg.items()` safely.

I've now done a thorough adversarial read of all in-scope files. Let me do one final check on a subtle point in `run\_phase3\_eval.py`: line 92 `db\_path = args.db` or `os.path.join(cfg.output.output\_dir, cfg.output.db\_name)` then line 93 `os.makedirs(os.path.dirname(db\_path) or ".", exist\_ok=True)`. If `args.db` is a bare filename like `foo.db`, `os.path.dirname` returns `""`, then `""` or ` "."` ? ` "."`. Good handling.

Let me verify one last concern in `harvest.py`: line 41 `ffmeta = (val\_context or {}).get("ffprobe\_meta") or {}`. Then `streams = ffmeta.get("streams") or []`. If `streams[0]` exists but is `None` (a null entry), line 45 `s = streams[0]` then `s.get(...)` ? `AttributeError`. But `streams[0]` being `None` is unusual ffprobe output. Low likelihood. The audio comprehension at line 74 guards `(s or {})` but line 45 does not guard `streams[0]`. This is a minor inconsistency ? line 74 defensively handles `None` entries but line 45 doesn't. However, ffprobe never emits null stream entries, so it's not realistically reachable. I'll note it as low confidence.

Let me verify the `\_to\_list` in export.py for a subtle correctness issue: `list(segments)` if not `isinstance(segments, list)` else `segments`. This returns the original list reference when already a list (no copy), which is fine for read-only use.

I have completed my adversarial review. The code in this area is well-hardened with explicit

defensive comments throughout (the ``unwrap_or_failure`` notes, the ``_read_time`` fallbacks, the never-empty safeguards). I found no high-confidence reachable defects. The two lowest-confidence observations (doctor.py always exits 0; harvest.py ``streams[0]`` not None-guarded while the audio loop is) are characteristics rather than clear defects and are not realistically reachable bugs.

Let me record my findings.

49. user (2026-06-18T07:50:48.275Z)

Structured output provided successfully

50. assistant (2026-06-18T07:51:01.863Z)

I completed a thorough adversarial read of every in-scope file and found no high-confidence reachable defects to report.

Files reviewed in full:

- ``scripts/nightly_regression.py`` ? exit-code ladder (0/1/2/3/4) matches the docstring; the gated metric sets (``GATED_HIGHER``, ``GATED_LOWER``, ``PER_VIDEO_GUARD``) and snapshot key lists (``AGG_SNAPSHOT``, ``PV_SNAPSHOT``) all align with the keys actually produced by ``backend/eval/rally_seg_eval.py`` ``score_intervals``/``aggregate``; ``summarize_run`` defensively extracts ``["mean"]`` and ``if k in r``; corpus/preset drift correctly degrades to STALE while still running the per-video intersection guard; atomic ``os.replace`` baseline write.
- ``scripts/doctor.py``, ``scripts/run_phase3_eval.py``, ``scripts/run_process_and_stitch.py`` ? path/cwd resolution is rooted off ``__file__``; ``os.path.dirname(db_path)`` or ``.`` handles bare filenames; ``unwrap_or_failure`` guards the Result/dataclass ``len()`` footgun.
- ``scripts/nightly_regression.ps1``, ``scripts/register_nightly_task.ps1`` ? graceful no-op guards, exit-code passthrough, codes 2/3 correctly fall to ERROR.
- ``backend/reporting/harvest.py``, ``run_signals.py``, ``spec.yaml`` ? ``r_frame_rate`` parsing wraps ``ValueError``/``ZeroDivisionError``; the ``segmenter_divergence`` rule gates on ``segmenter_explicitly_requested`` so the unconditional ``matches_default`` computation is harmless; thread-locked in-process signal store.
- ``backend/tools/export.py``, ``rally_slicer.py`` ? ``_segment_metadata`` handles inline-dict vs JSON-string (DB round-trip yields dict via ``query_play_segments``, confirmed); EDL frame field stays in 0?29; ``load_rally_labels`` skips degenerate rows; ``compute_clip_windows`` clamps and drops past-end windows.
- ``training/gen0/harness.py`` ? sign-test p-value math is the correct one-sided binomial tail (cross-checked against ``tests/test_training_gen0.py::test_sign_test_math``); floor ``per_video`` dict access matches the documented shape; ship-gate is correctly hard-wired ``ship=False``.

Two lowest-confidence observations I deliberately did not report as defects because they are not realistically reachable: ``scripts/doctor.py`` never sets a non-zero exit even when FFmpeg checks FAIL (it is a setup/bootstrap helper, not a gate), and ``backend/reporting/harvest.py`` line 45 dereferences ``streams[0].get(...)`` without the ``(s or {})`` None-guard that the audio loop at line 74 uses (ffprobe does not emit null stream entries).